

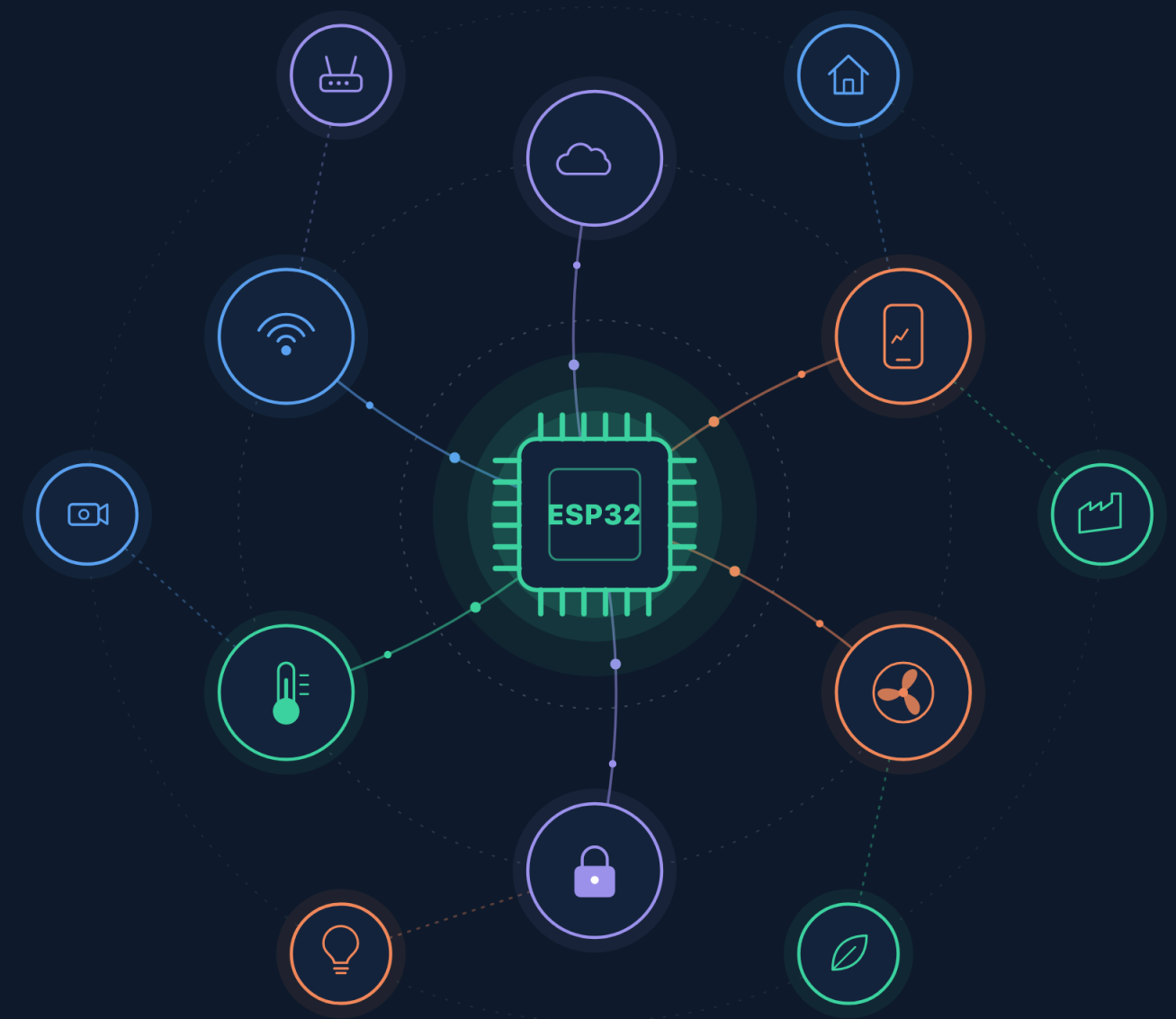
MODULE 1 • 45 MINUTES

What Is the Internet of Things?

Where it came from, what makes a device an IoT device, and where you meet it every day.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

You are here

02

How it's built

Lecture

03

Sensors and actuators

Labs 1–2

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Explain** what the Internet of Things is, in your own words
 - 02 **Describe** how the internet grew from computers to everyday objects
 - 03 **List** the characteristics that make something an IoT device
 - 04 **Explain** why IoT is growing so fast
 - 05 **Give examples** from six areas: homes, cities, industry, health, farms, transport

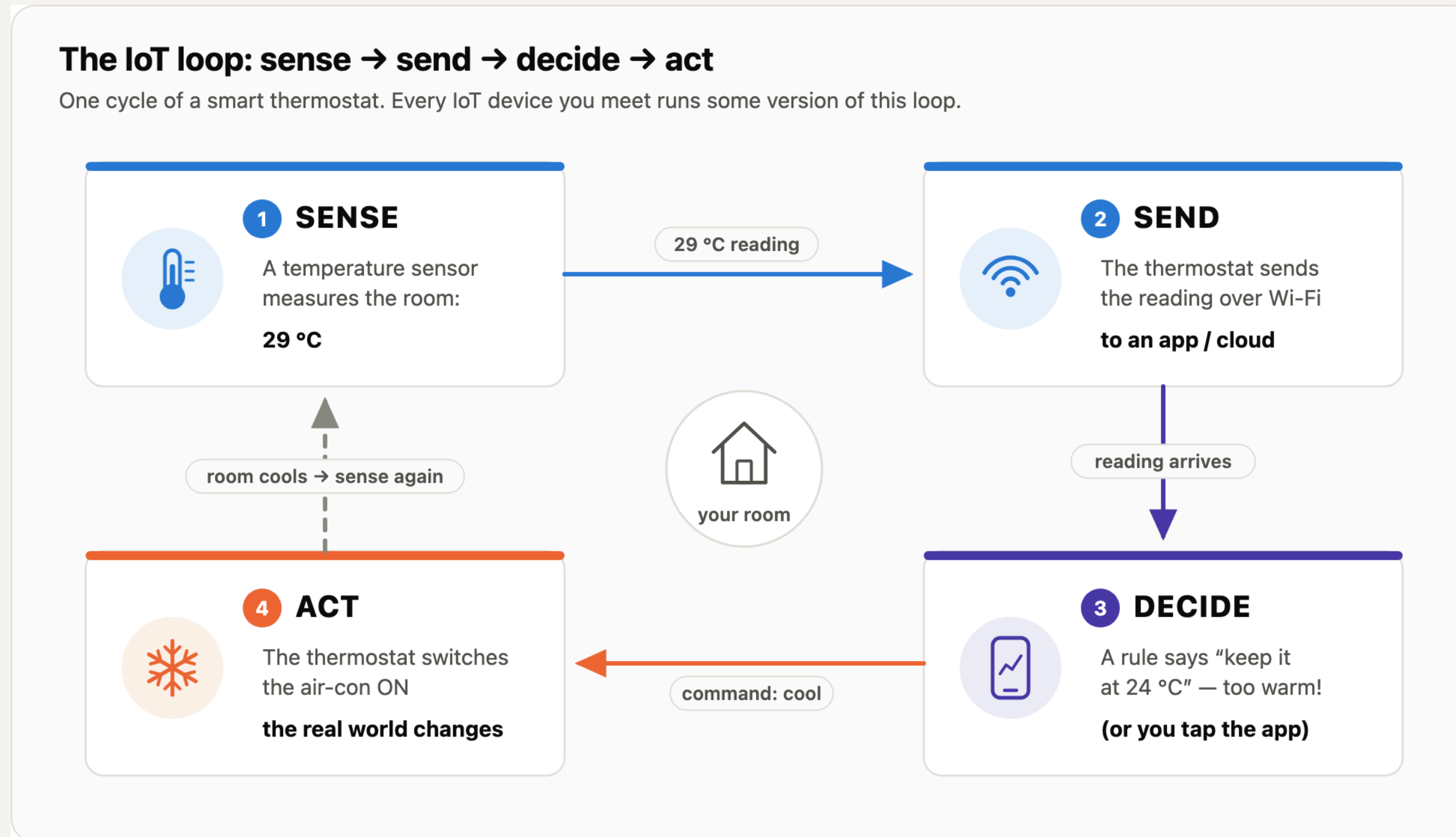
A SIMPLE DEFINITION

The Internet of Things is the network of physical objects that sense the world, share what they sense, and act on it.

A “thing” can be almost anything: a light bulb, a water pump, a streetlight, a heart monitor, or a cow with a smart collar.

THE IOT LOOP

Sense, send, decide, act





A REAL DEVICE

A smart thermostat

- **Sense:** a temperature sensor inside
- **Send:** Wi-Fi to an app
- **Decide:** a rule, or you
- **Act:** switches heating or cooling

PAUSE AND CHECK

A smart doorbell has a camera and a button. Someone presses it, you get a video call, and you unlock the door from your phone. Can you find the four steps?

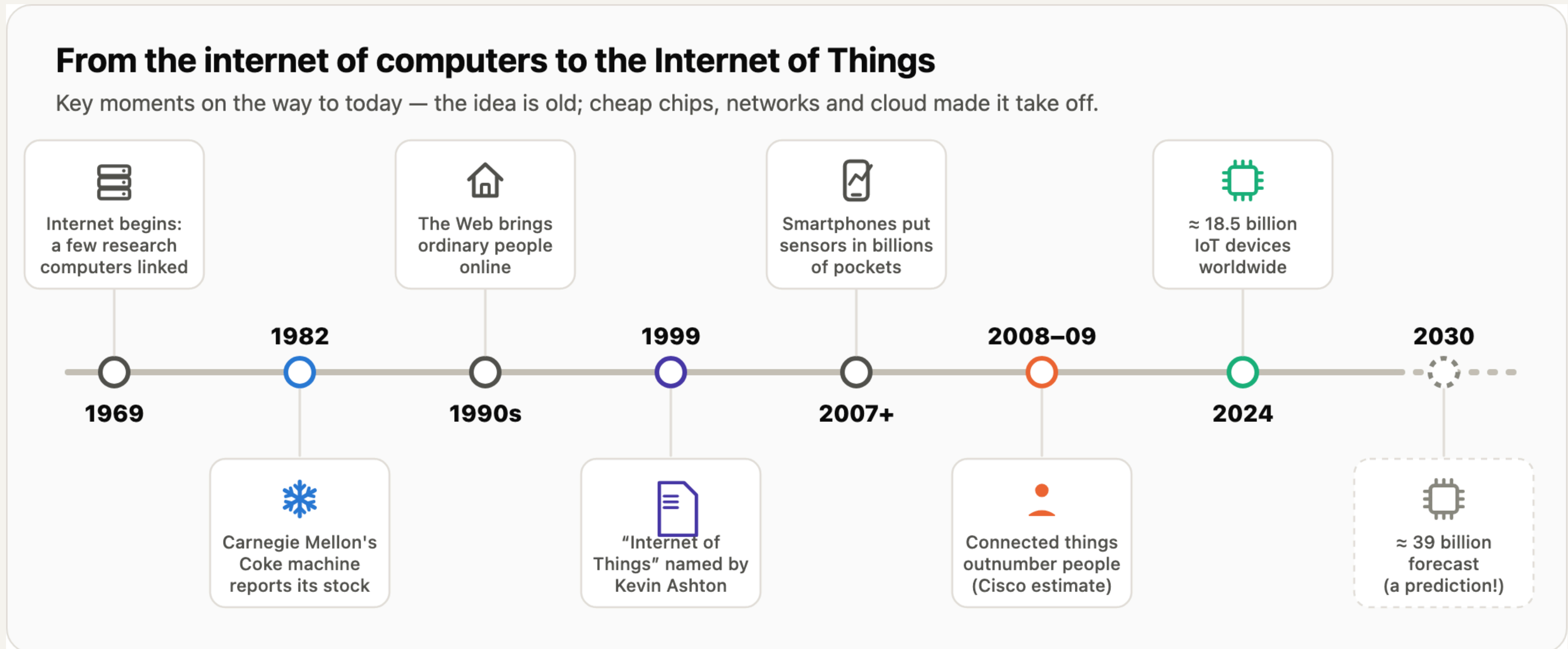
Sense → send → decide → act. Talk to your neighbour for one minute.

Part A

How we got here

From an internet of computers to an Internet of Things.

From a few computers to billions of things





1999 · A NAME IS BORN

Kevin Ashton names the idea

Working on radio tags (RFID) for products, he asked: what if things could report on themselves, instead of people typing in the data?

WHAT MAKES A DEVICE AN IOT DEVICE?

Eight shared characteristics

Characteristic	What it means	Example
Sensing	Measures something in the physical world	A smart meter measures electricity
Connectivity	Sends and receives data over a network	The meter reports to the power company
Unique identity	Each device can be told apart	Each meter has its own ID
Automation	Processes data and acts by itself	A smart plug turns off a heater
Ability to act	Changes the physical world	A smart valve shuts off a leak
Scale	Often huge numbers of devices	A city’s thousands of streetlights
Limited resources	Little power, memory and processing	A soil sensor on one battery for years
Variety	Many makers, still working together	Lights and locks from different brands

If you remember only two: it **senses or acts** on the physical world, and it is **connected**.

Part B

Why now?

The Coke machine was 1982. So why has IoT exploded only in the last fifteen years?

FOUR THINGS GOT CHEAP AT ONCE

Why IoT is growing so fast



Sensors

Smartphones created a huge market for tiny sensors, so prices fell for everyone.



Chips

A microcontroller with Wi-Fi built in, like the ESP32, now costs very little.



Connectivity

Wi-Fi and mobile networks are almost everywhere, even in remote fields.

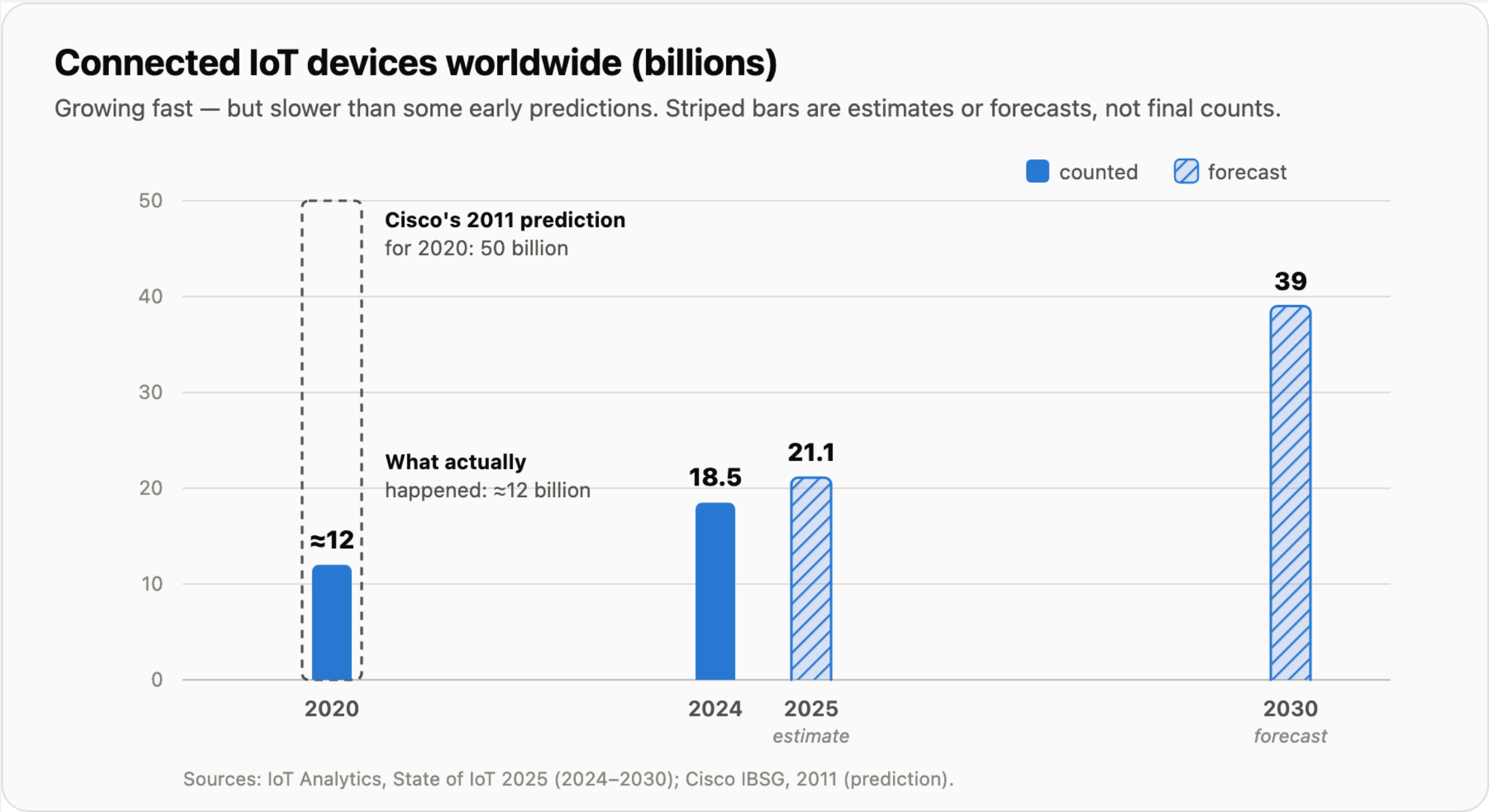


The cloud

Companies rent storage and computing by the minute, instead of buying servers.

HOW BIG IS IT?

Billions of connected devices



18.5 billion IoT devices in 2024

39 billion forecast for 2030

Most connect by **Wi-Fi**, then Bluetooth and cellular

Source: IoT Analytics. Dashed outline: Cisco’s 2011 forecast for 2020.

TREAT FORECASTS WITH HEALTHY DOUBT

In 2011, Cisco predicted 50 billion devices by 2020. The real number was about 12 billion.

When you read a big number, ask: who made it, and how did they count?

WHAT'S HAPPENING NOW

Five trends shaping IoT



Smarter edge

Devices decide for themselves; some run small AI models (AIoT).



Matter

A smart-home standard so brands work together.



New networks

LoRaWAN and NB-IoT reach sensors for years on a battery.



Security laws

New rules ban weak default passwords.



Sustainability

Saving energy and water, like dimming empty streets.

Part C

Where is IoT used?

Six application areas — and some examples you probably know.

SIX APPLICATION AREAS

IoT is almost everywhere



Smart homes

Comfort, safety and saving energy.



Smart cities

Streetlights, parking, traffic and air quality.



Industry (IIoT)

Fix machines before they break.



Healthcare

Wearables and patients monitored at home.



Agriculture

Water only where and when crops need it.



Transport

Know where things are and how they are doing.



SMART CITIES

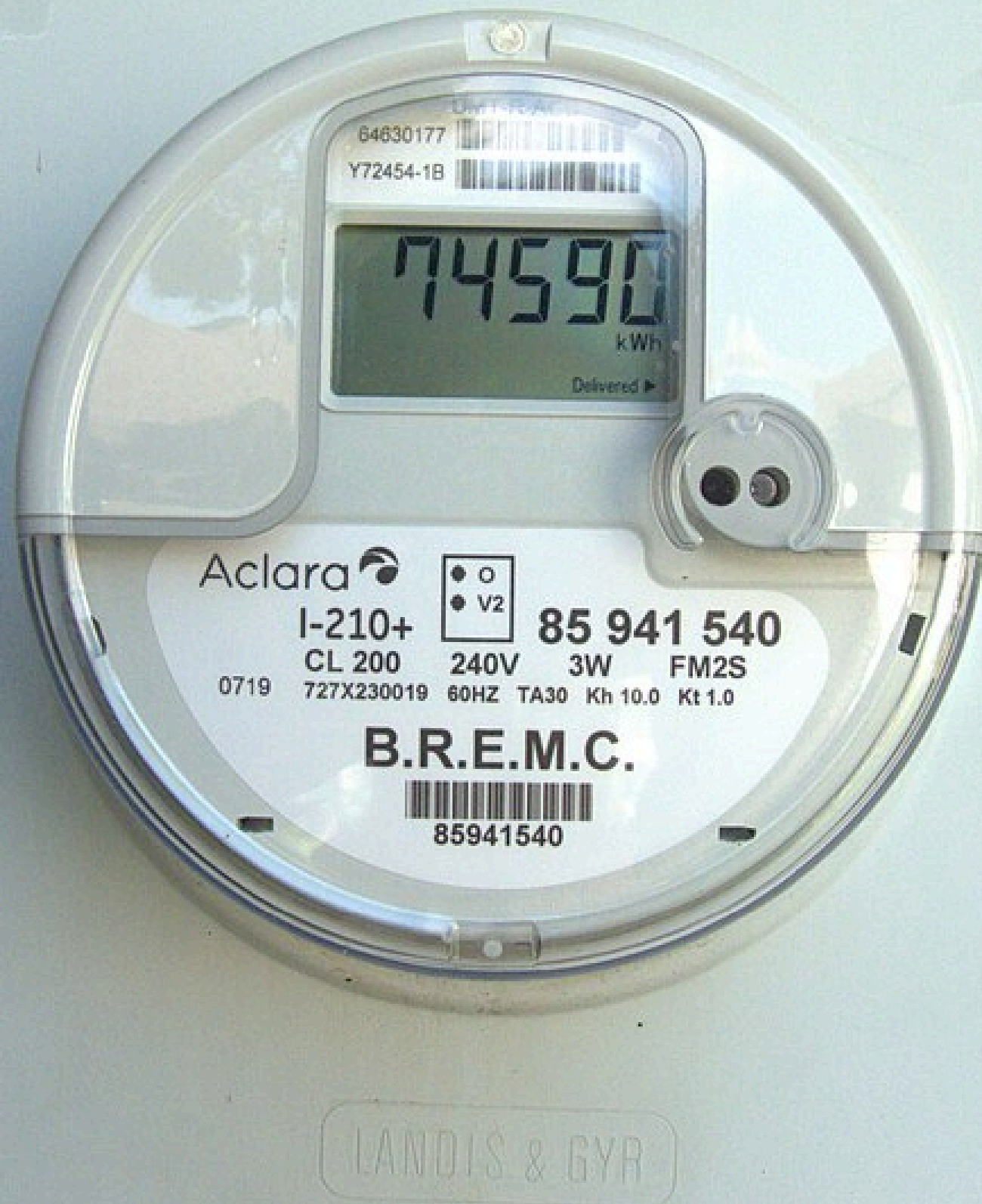
Streetlights that report in

- Los Angeles connected tens of thousands of LED streetlights
- About **64% less energy** than the old lights
- Broken lights report themselves straight away
- Putrajaya runs its own LoRaWAN network for services like smart parking

AN EXAMPLE YOU PROBABLY KNOW

Smart electricity meters

- **Sense** electricity use
- **Send** readings to TNB automatically
- **Decide** the bill, or where a power cut is
- **Act**: crews fix faults; customers save energy
- 5.6 million+ TNB customers already have one



COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *IoT means controlling things from your phone.*

✓ Fact Most IoT data never reaches a phone. Other computers use it.

✗ Myth *IoT is just gadgets for the home.*

✓ Fact Industry, cities, farms, health and logistics are a huge share.

✗ Myth *An IoT device must be powerful.*

✓ Fact Most are tiny, cheap and low-power — on purpose.

✗ Myth *IoT is a single technology.*

✓ Fact It combines sensors, chips, networks, cloud, data and security.

Where did you meet IoT today?

- 1 List every connected device you used or passed since you woke up.
- 2 For each one, find the loop: sense → send → decide → act.
- 3 Sort them into the six areas: home, city, industry, health, farm, transport.
- 4 Pick one and ask: what data does it collect about me, and who can see it?

KEY TAKEAWAYS

What to remember

- ✓ IoT = physical objects that **sense, share and act** on data
- ✓ Every IoT device follows the loop: **sense → send → decide → act**
- ✓ Named in **1999**; things outnumbered people around **2008–09**
- ✓ Growing fast because sensors, chips, networks and cloud got **cheap**
- ✓ Six areas: **homes, cities, industry, health, farms, transport**

Quick quiz

Q1 A normal scale shows the weight on its screen. A smart scale sends it to an app. Which one is IoT — and why?

Q2 Apply the four steps to an automatic garden sprinkler.

Q3 Give two reasons why IoT has grown so quickly.

Q4 Which area? (a) an overheating factory motor (b) a GPS tracker on a lorry (c) a parking sensor

UP NEXT • MODULE 2

How an IoT System Is Built

We open the “black box”: the five layers of every IoT system, and the journey of one reading from a sensor to a dashboard.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

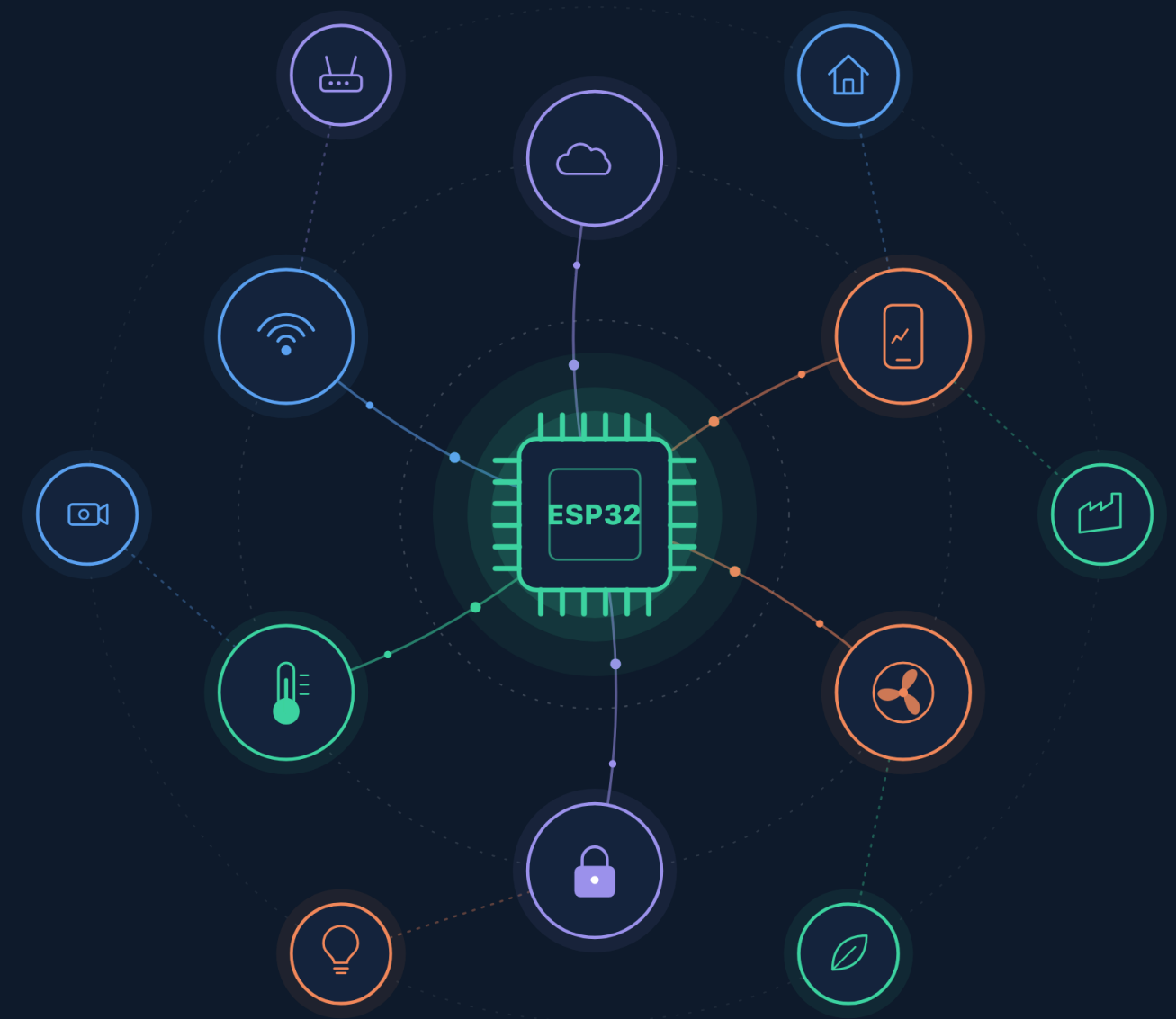
MODULE 2 • 45 MINUTES

How an IoT System Is Built

The five layers inside every IoT system, the small computers that run devices, and the journey of one reading.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

You are here

03

Sensors and actuators

Labs 1–2

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Name** the five layers of an IoT system and what each one does
 - 02 **Explain** the difference between a sensor and an actuator
 - 03 **Compare** a microcontroller with a microprocessor
 - 04 **Trace** one reading from a sensor to a person's screen
 - 05 **Identify** the main groups who make an IoT product happen
-

WHERE WE ARE

Module 1 showed what an IoT system does.
Module 2 shows how it is put together.

Think of lifting the bonnet of a car to see the engine, the wheels and the steering.

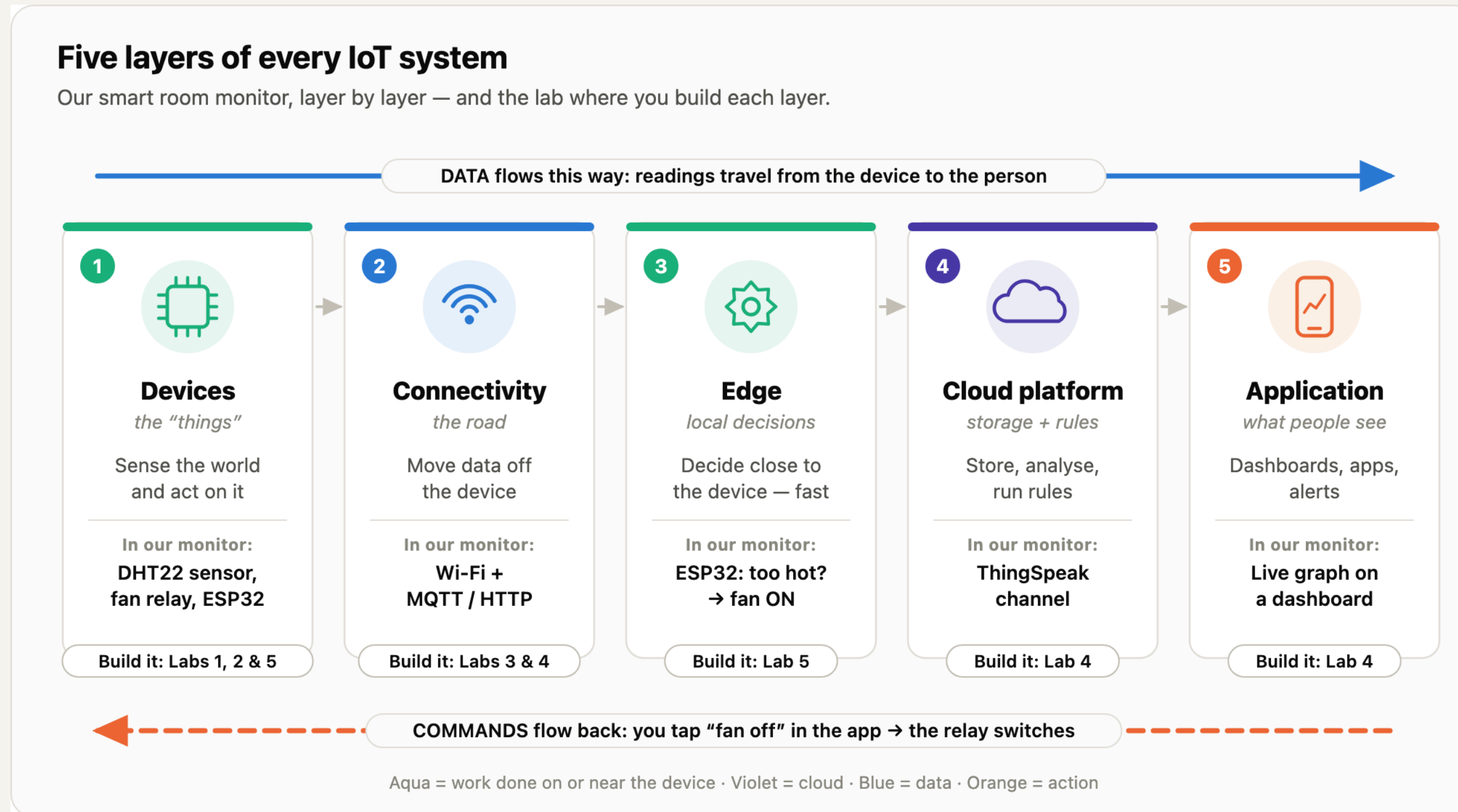
Part A

The five layers

One smart room monitor, broken down layer by layer.

THE BIG PICTURE

Five layers of every IoT system



Three ingredients in every device



Sensors

Measure something in the real world, like temperature.



Actuators

Do something in the real world, like switching on a fan.



A small computer

Usually a microcontroller that runs the program.

In the labs, your device is a simulated **ESP32** board.

The road the data travels on



The radio

How the signal travels, like Wi-Fi or Bluetooth.



The language

What the device speaks, like MQTT or HTTP.



A gateway

A middleman box that serves many nearby devices.

Your home Wi-Fi router is a kind of gateway. So is a smart-home hub.

LAYER 3 · EDGE COMPUTING

Edge computing means doing some of the thinking close to where the data is collected.

On the device itself, or on a gateway nearby — not far away in the cloud. It is faster, and it keeps working when the connection doesn't.

What a cloud platform does



Stores

Keeps old readings,
so you see history.



Processes

Works things out,
like a daily average.



Runs rules

“Above 28 °C? Send
me a message.”



Manages devices

Knows which
devices exist and
may connect.

In Lab 4 you use a free cloud platform, **ThingSpeak**. Module 5 covers this layer.

LAYER 5 · APPLICATION

A farmer doesn't care about data centres. They care that their phone says “Field 3 needs water”.

Dashboards, phone apps, email alerts, reports — or another computer system, like billing.

PAUSE AND CHECK

Our room monitor switches the fan on by itself when it gets hot, without asking the cloud. Which layer is making that decision?

Devices · Connectivity · Edge · Cloud platform · Application

Part B

Inside the device

Senses, muscles and a brain.

Sensors and actuators

	Sensor (input)	Actuator (output)
Job	Measures the world	Changes the world
Like your...	Senses	Muscles
Examples	Thermometer; light, motion and gas sensors	LED, buzzer, motor, relay (an electric switch)
In our monitor	Temperature and humidity sensor	Relay that switches the fan

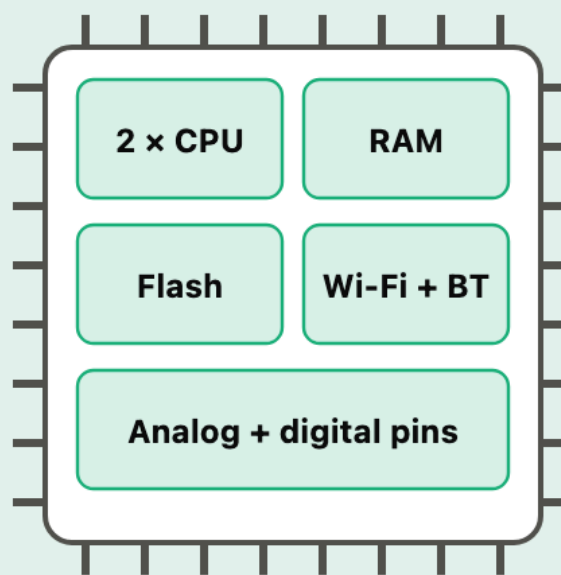
Ask: **which way is the information going?** In from the world = sensor. Out into the world = actuator.

One chip, one job vs a small computer

One chip, one job vs a small computer

A microcontroller packs everything onto one chip. A microprocessor needs a board of helpers and an OS.

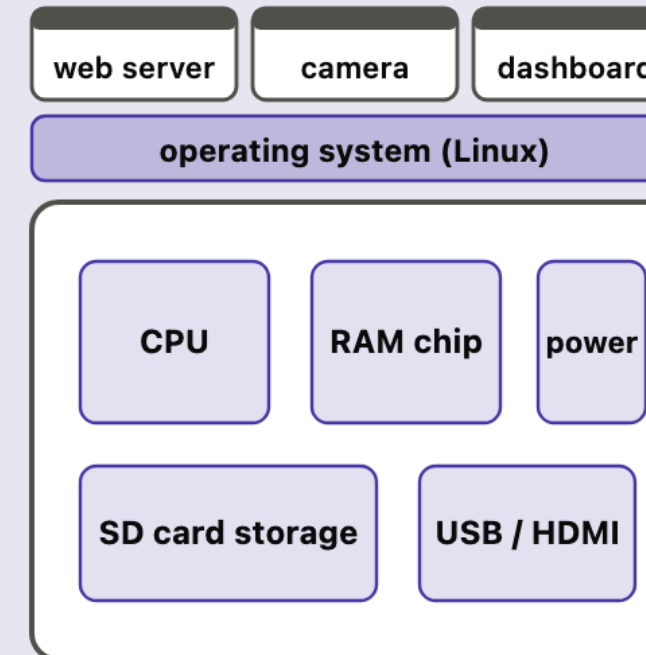
MICROCONTROLLER (e.g. ESP32)



everything on ONE chip

- ✓ **Runs ONE program,**
from the moment power is on
- ✓ **Starts instantly**
no booting
- ✓ **Deep sleep $\approx 10 \mu\text{A}$**
months–years on batteries
- ✓ **Low cost**
cheap enough to put in anything

MICROPROCESSOR BOARD (e.g. Raspberry Pi)



several chips on a board

- i **Runs an OS +**
many programs at once
- i **Boots in seconds**
like a small PC
- i **Draws watts**
usually mains-powered
- i **Great for video, AI**
and heavy software

SIDE BY SIDE

Microcontroller or microprocessor?

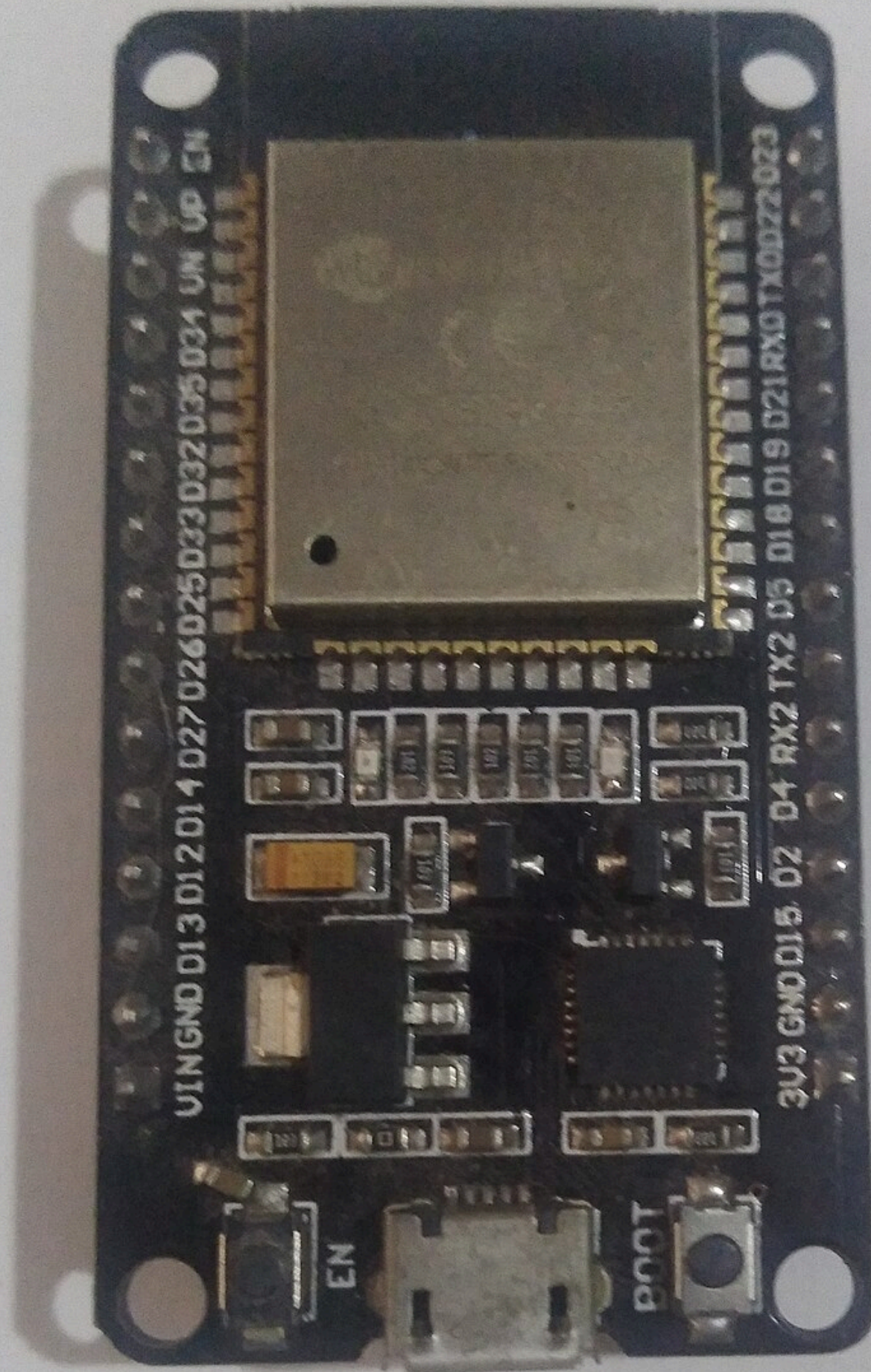
	Microcontroller (MCU)	Microprocessor (MPU)
Like a...	Kitchen timer: one job, done reliably	Laptop: many jobs, more looking after
Runs	One program, straight away	An operating system and many programs
Power use	Very low: months or years on batteries	Higher: usually mains power
Start-up	Almost instant	Seconds to a minute (it boots)
Cost	A few US dollars	Tens of US dollars and up
Good at	Reading sensors, switching things	Video, heavy data, a local dashboard
Examples	ESP32, Arduino boards, STM32	Raspberry Pi, the chip in your phone

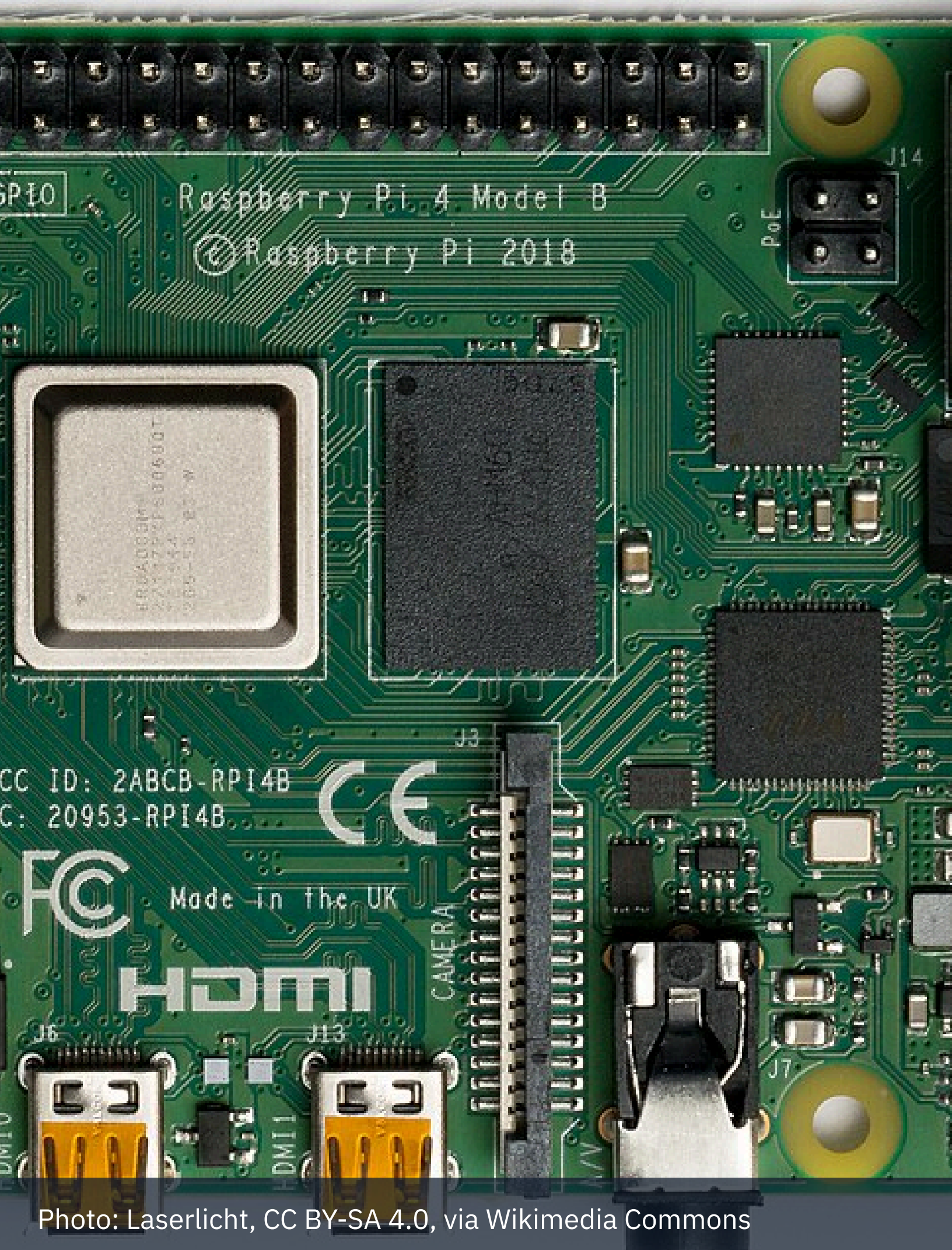
Reads sensors and flips switches? **Microcontroller.** Video or lots of software? **Microprocessor.**

THE DEVICE YOU'LL USE

Meet the ESP32

- **Wi-Fi and Bluetooth** built into the chip
- **Cheap:** a board costs very little
- **Capable:** 2 cores, 240 MHz, 520 KB memory
- **Deep sleep:** the chip uses about 10 microamps
- **Lots of pins** for sensors and actuators





THE OTHER FAMILY

Meet the Raspberry Pi

- Built around a **microprocessor**
- Runs a full **operating system**
- Connects a camera or a screen easily
- Can be the **gateway** for many sensors

Part C

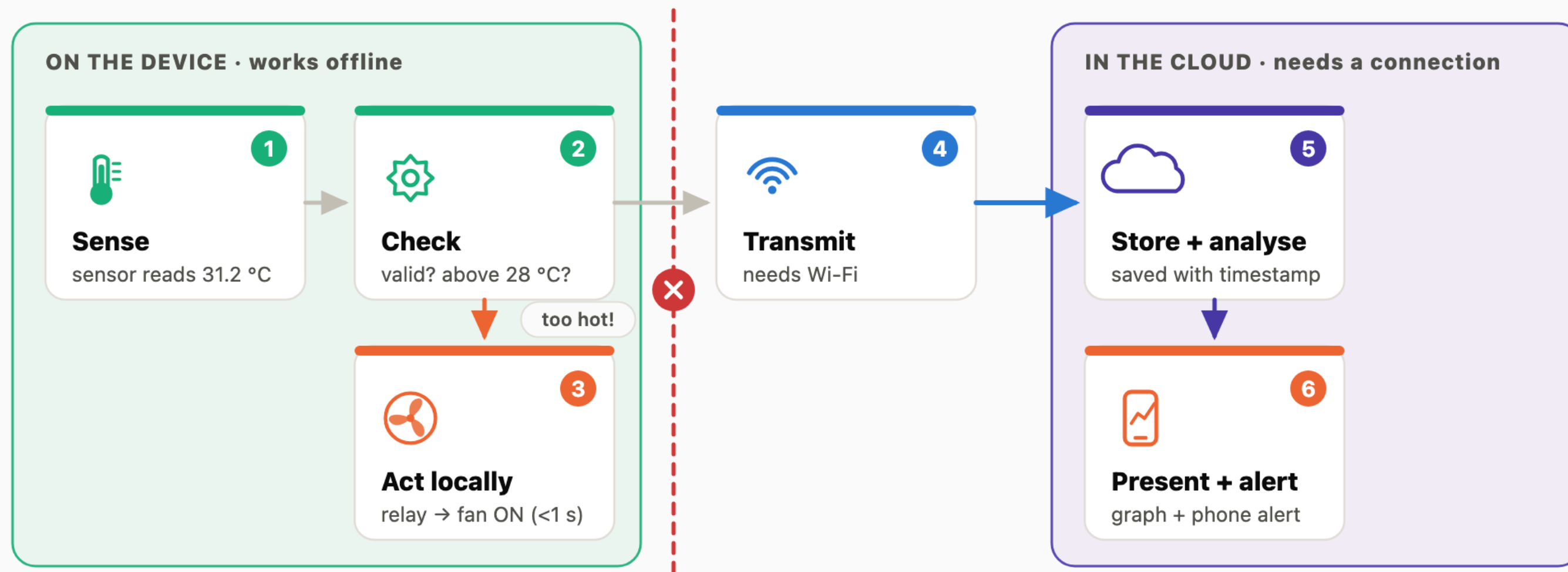
Following one reading

From a sensor, through the layers, to a person — and who makes it all happen.

One reading's journey

Following one reading through the pipeline

31.2 °C is measured. Steps 1–3 happen on the device before any data leaves it.



If Wi-Fi drops, steps 1–3 (left of the red line) still run — the fan still switches on

PAUSE AND CHECK

In the pipeline, which steps would still work if the Wi-Fi went down?

1 Sense · 2 Check · 3 Act locally · 4 Transmit · 5 Store · 6 Present. Talk to your neighbour for one minute.

Who makes it all happen

Group	What they do	Examples
Chip makers	Design the chips and sensors	Espressif, Nordic, STMicroelectronics
Device makers	Build the product people buy	Smart-plug and wearable brands
Connectivity providers	Provide the network	Mobile, LoRaWAN and internet providers
Cloud and platforms	Store data and run rules	AWS, Microsoft, ThingSpeak, Blynk
Developers and integrators	Build apps, connect everything	Software companies, system integrators
End users	Use the system and pay for it	Home owners, farmers, hospitals, cities
Regulators	Set rules on safety, radio, privacy	Telecoms regulators, standards bodies

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *An IoT device must connect directly to the internet.*

✓ Fact Many talk only to a nearby gateway, which reaches the internet.

✗ Myth *The cloud does all the thinking.*

✓ Fact More and more decisions happen at the edge: faster, and offline too.

✗ Myth *A Raspberry Pi and an Arduino are the same thing.*

✓ Fact Different families: an OS on a microprocessor vs one program.

✗ Myth *More layers means a more advanced system.*

✓ Fact Three or seven layers can describe exactly the same system.

Map a smart office building

- 1 Draw five boxes: devices, connectivity, edge, cloud, application.
- 2 Put one part of a smart office into each box.
- 3 Which decision should happen at the edge? Why?
- 4 Who sees the result, and what do they see?

KEY TAKEAWAYS

What to remember

- ✓ Five layers: **devices, connectivity, edge, cloud, application**
- ✓ Data flows **up** to people; commands flow back **down**
- ✓ Sensors bring information **in**; actuators push action **out**
- ✓ **Microcontrollers** do one job cheaply; **microprocessors** run an OS
- ✓ A reading follows a **pipeline**, and urgent decisions stay at the edge

Quick quiz

Q1 List the five layers, in order from the device to the person.

Q2 In a smoke detector, is the siren a sensor or an actuator? And the part that detects the smoke?

Q3 A soil sensor must run for a year on a small battery. Which brain: a microcontroller or a microprocessor?

Q4 A smart doorbell stops showing video in its app. Name three places where the fault could be.

UP NEXT • MODULE 3

Sensors, Actuators, and Reading Data

We zoom in on Layer 1: how sensors make their signals, analog vs digital, and your first circuit in Lab 1.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

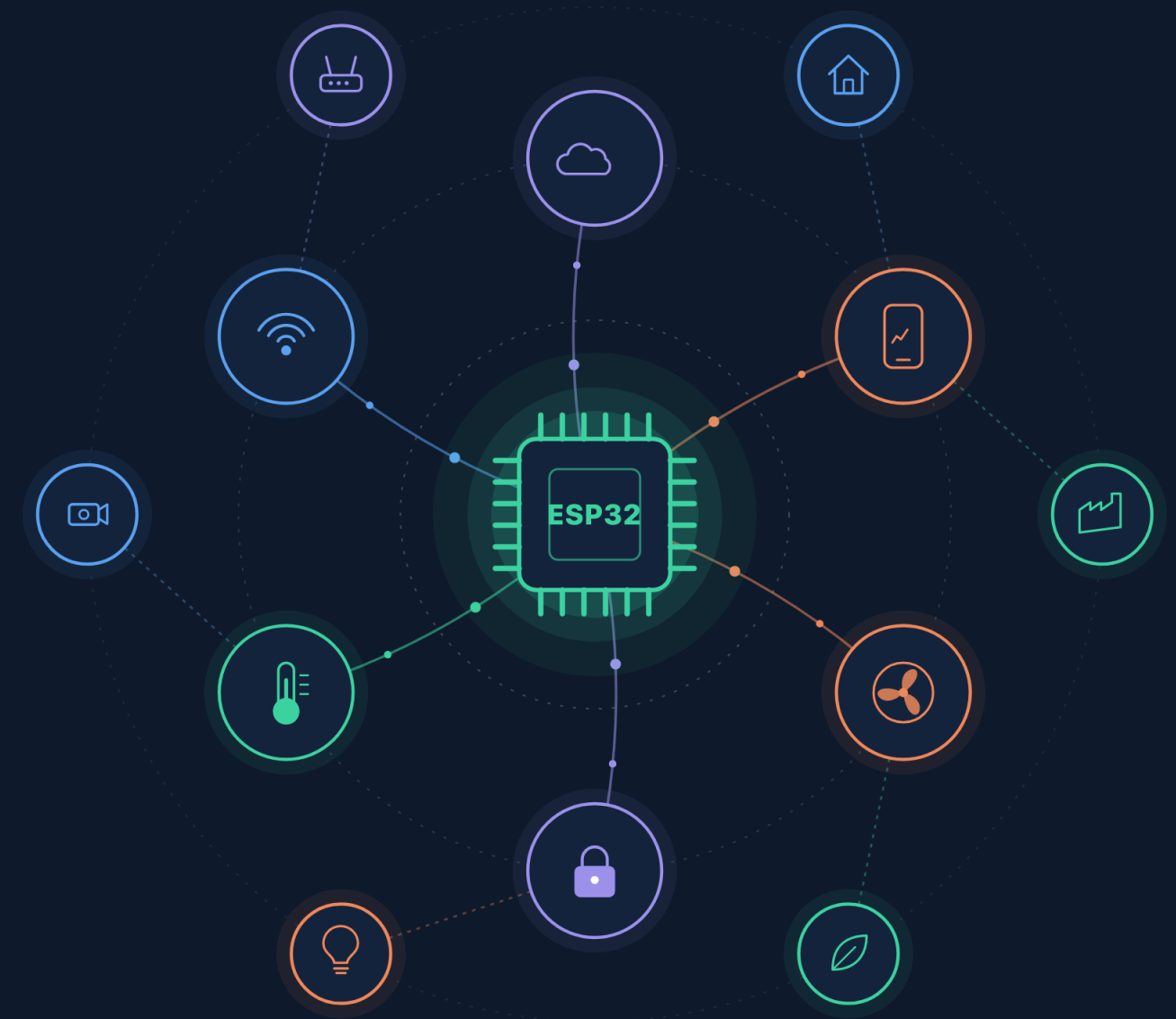
MODULE 3 • 45 MINUTES

Sensors, Actuators, and Reading Data

How a device feels the room, switches on a fan, and turns signals into numbers.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

You are here

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

01 Describe five common sensors and four common actuators

02 Explain the difference between analog and digital signals

03 Explain how a microcontroller turns a signal into a number

04 Read a short Arduino program and say what each part does

05 Connect and read a real sensor in the Wokwi simulator

THE BIG PICTURE

The journey of one reading



Real world

The room is 26 °C.



Sensor

Turns heat into an electrical signal.



Pin

The microcontroller reads the signal.



Your code

Converts the signal into a number.



26.0 °C

Ready to use or to send.

Actions go the other way: code decides → a pin sends a signal → an actuator acts.

Part A

Sensors: how devices feel the world

Five common types, and what each one measures.

FIVE COMMON SENSORS

What each sensor measures



Temperature

How hot or cold. A thermistor, for example.



Humidity

Water vapour in the air, as a percentage.



Light

How bright. A photoresistor, or LDR.



Motion (PIR)

A warm body moving past.



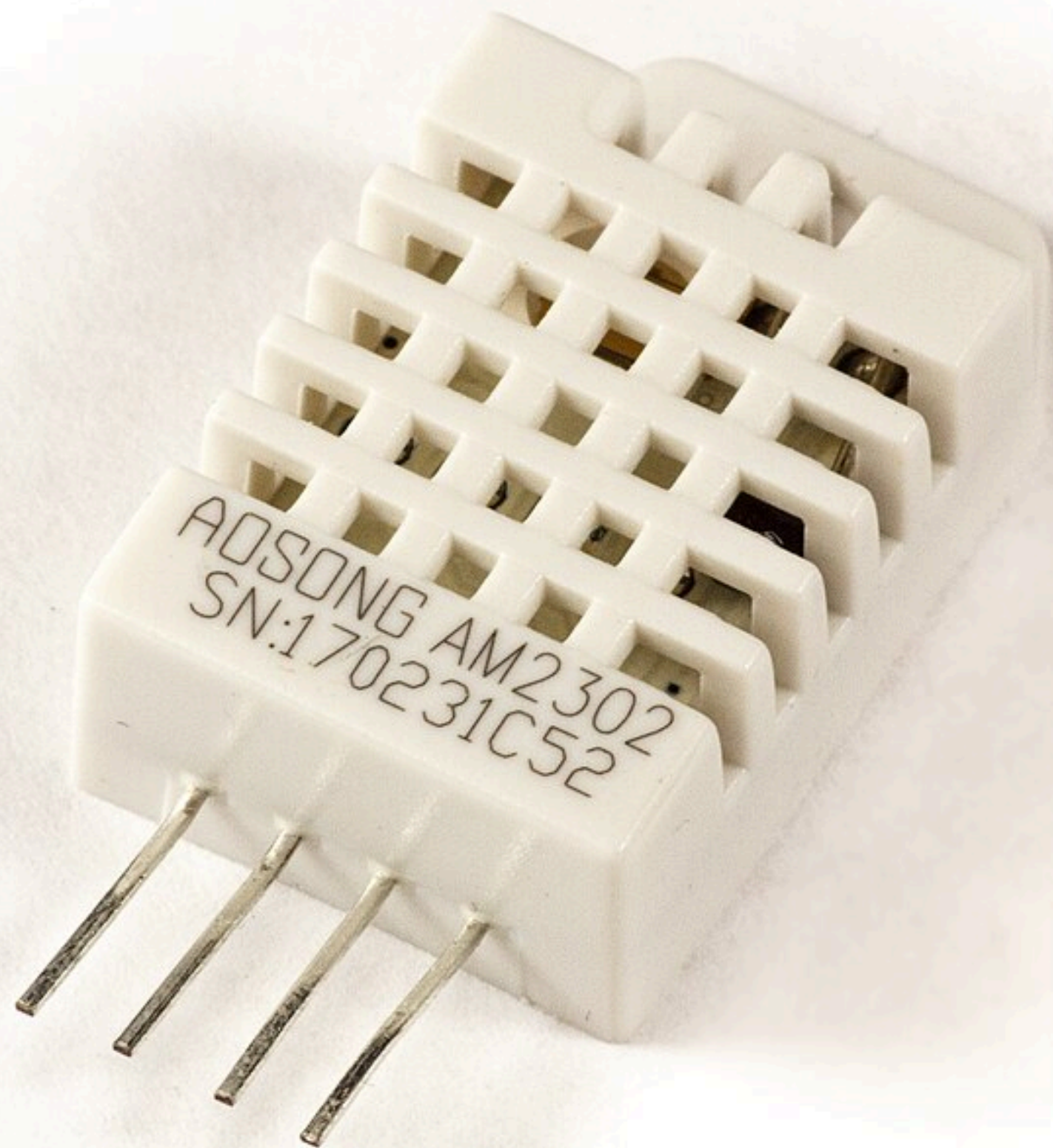
Gas

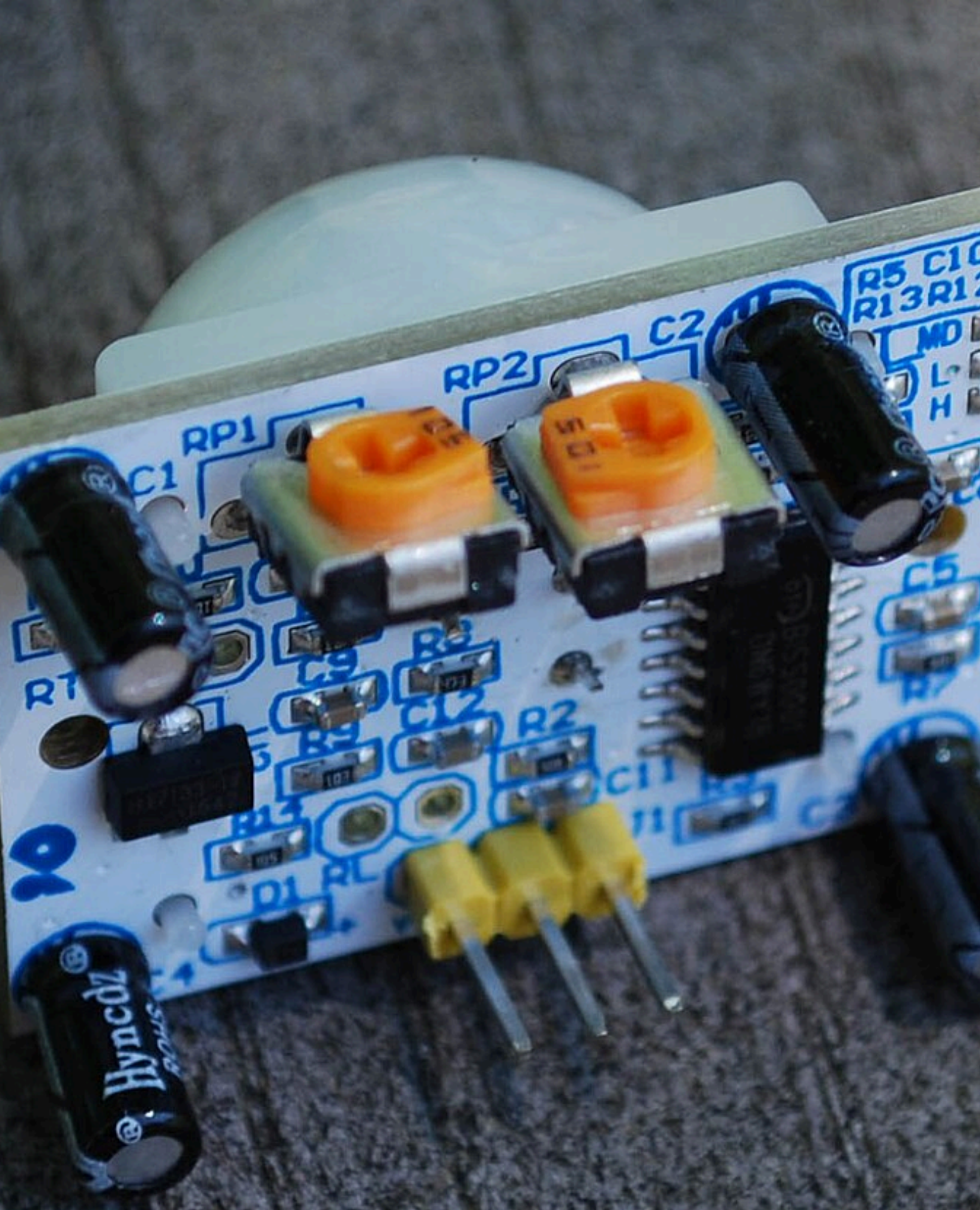
Gases like carbon monoxide or LPG.

TWO SENSORS IN ONE

The DHT22

- Temperature **and** humidity in one small part
- Also sold as the AM2302
- The sensor you will use in **Lab 2**





MOTION SENSOR

A PIR motion sensor

- Everything warm gives off **infrared** light
- It notices when that warmth pattern **changes**
- Two dials: sensitivity, and how long it stays on
- Security lights, automatic doors, alarms

Part B

Actuators: how devices act

Light, sound, switching and movement.

FOUR COMMON ACTUATORS

From a signal to an action



LED

A small light. Needs a resistor. Great for status lights.



Buzzer

A beep or a tone, to get attention.



Relay

A switch for a bigger circuit: a fan, a pump.



Motor

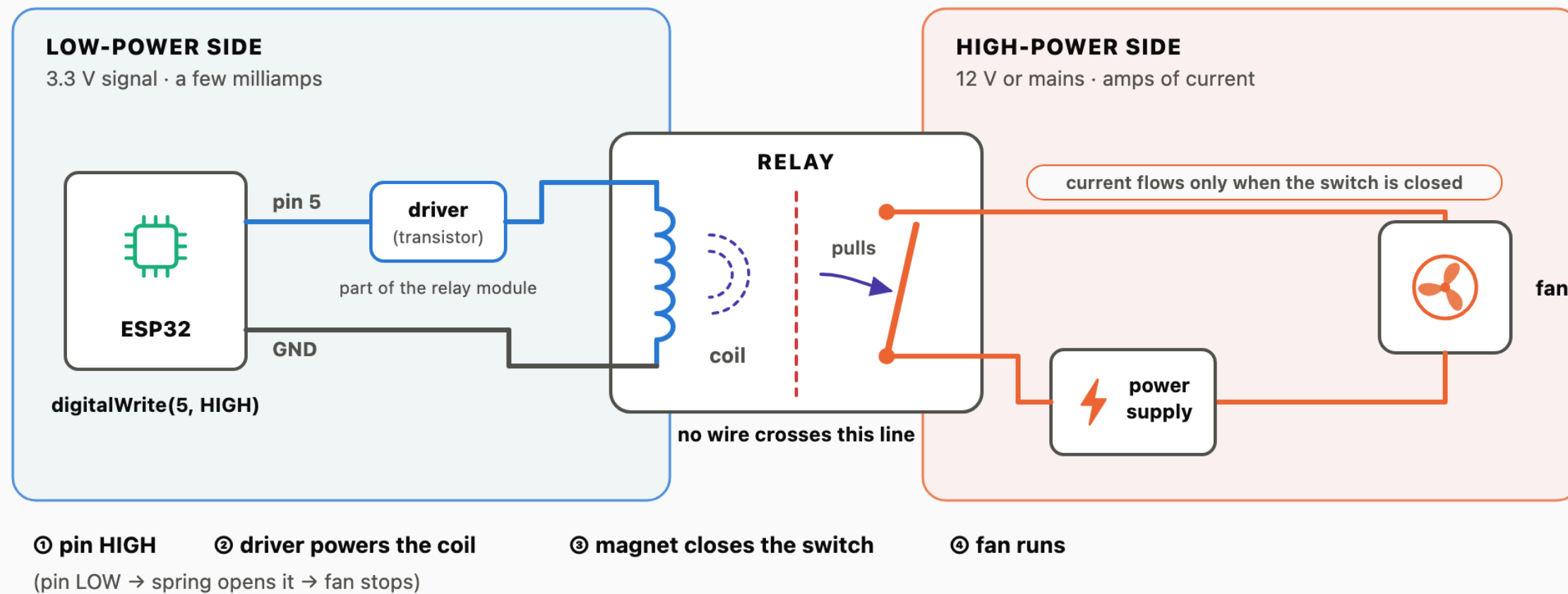
Movement. DC, servo or stepper, via a driver.

HOW A RELAY WORKS

A small signal switches a big circuit

How a relay lets a tiny chip switch a big load

Two separate circuits. The small one energises a coil; its magnetism closes the big one's switch.



A pin gives only a **small** amount of power

The coil's **magnetism** closes the switch

The two circuits stay **separate**

A REAL PART

A relay module

- A typical **5 V** relay module
- Screw terminals connect the high-power circuit
- Fans, pumps, heaters, lights
- **Mains electricity can kill.** The labs are simulated.



PAUSE AND CHECK

You want your room monitor to open a window vent to exactly half-way. Which actuator would you choose?

LED, buzzer, relay, or motor — and which kind of motor?

Part C

Analog vs digital signals

The most important idea in this module — and simpler than it sounds.

TWO KINDS OF SIGNAL

Digital and analog, side by side

	Digital	Analog
Values	Two: HIGH/LOW, 1/0, on/off	Any value in a range
Like a...	Light switch	Dimmer switch, volume knob
Sensors	Push button, PIR sensor	Potentiometer, light sensor, thermistor
Read with	digitalRead()	analogRead()
It answers	“Is it happening?”	“How much?”

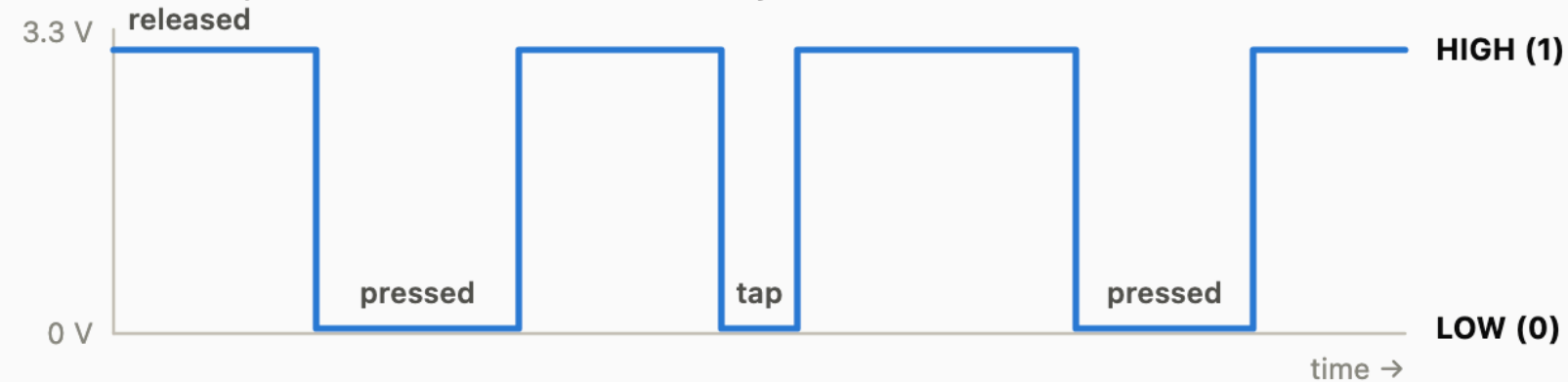
SEEING THE DIFFERENCE

From a voltage to a number

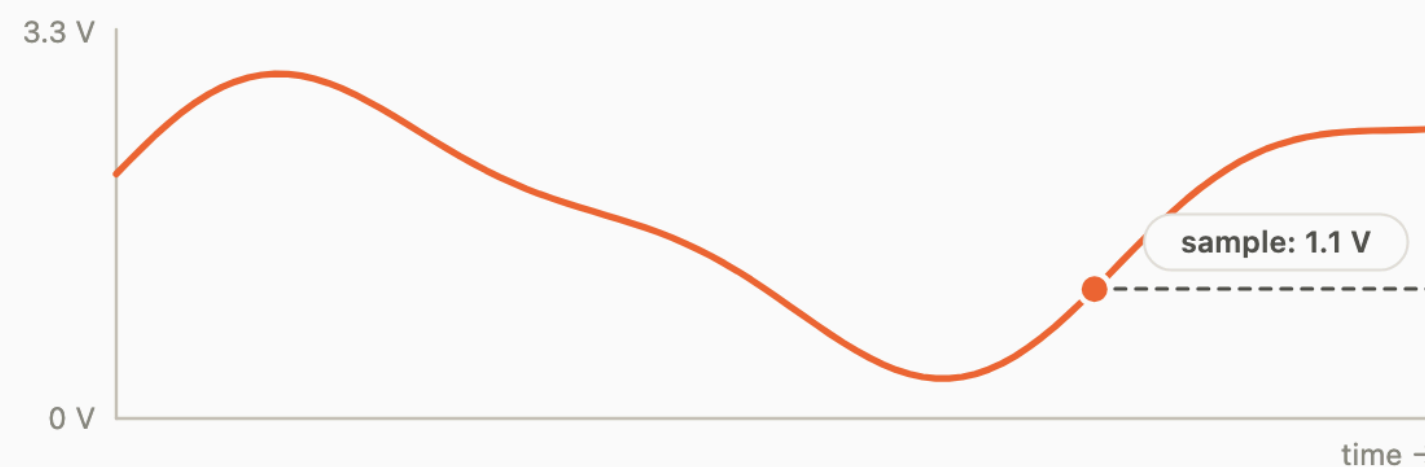
Digital vs analog — and how the ESP32 reads analog

Digital has two values. Analog can be anything in a range; the ADC measures it like a ruler.

DIGITAL a push button wired as in Lab 1: only HIGH or LOW

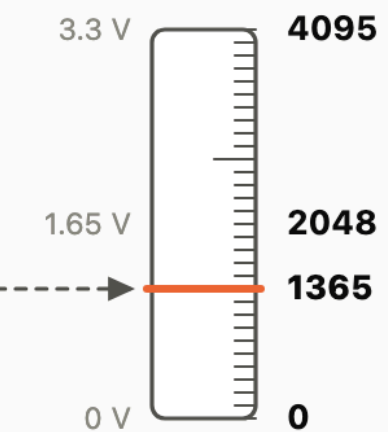


ANALOG a knob / light sensor: any value in between



THE ADC "RULER"

12-bit → 4,096 marks



then your code does:
analogRead() → 1365
 $1365 \div 4095 \times 100$
= 33.3 %

THE ESP32'S 12-BIT ADC

`analogRead()` gives 0 to 4095

0

at 0 V (nothing)

≈ 2048

at about 1.65 V (half-way)

4095

at about 3.3 V (full)

Read a raw number, then **convert** it: $(\text{raw} \div 4095) \times 100 = \text{percent}$. Use **ADC1 pins (GPIO 32–39)** with Wi-Fi.

Six rules for connecting parts



Three wires

Power, ground and signal.



Match voltage

The ESP32 works at 3.3 V.



No floating

Use a pull-up resistor on inputs.



Use a library

Ready-made code for smart sensors.



Respect limits

DHT22: one reading every 2 seconds.



Smooth noise

Average several readings.

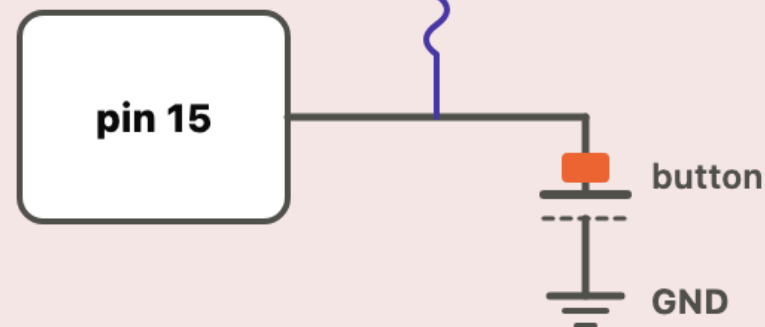
RULE THREE

Don't leave inputs floating

Why buttons need a pull-up resistor

An unconnected input "floats" and reads random noise. A pull-up holds it steady until pressed.

❌ **Floating input — no pull-up**
picks up stray noise
like an antenna

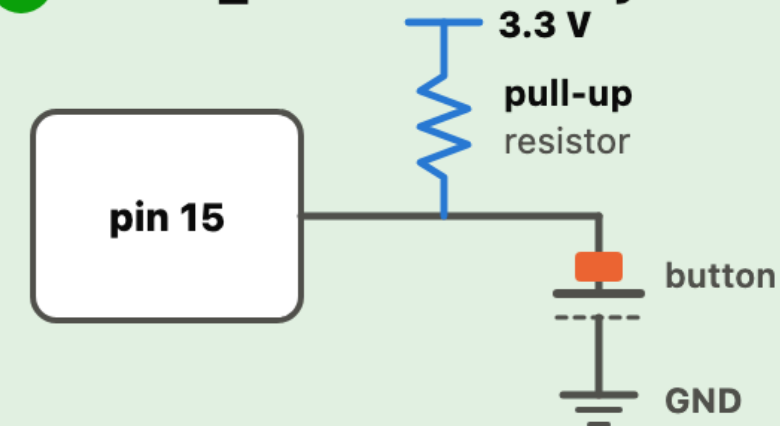


What digitalRead() returns, reading after reading:

H L H H L H L L H L H H L H

Random — the code sees presses that never happened.

✅ **INPUT_PULLUP — steady and reliable**



What digitalRead() returns, reading after reading:

H H H H H L L L L H H H H H

button held down

HIGH at rest, LOW only while pressed.

Code: `pinMode(15, INPUT_PULLUP);`

PAUSE AND CHECK

Your button randomly shows “pressed” even when nobody is touching it. What’s the most likely cause, and the fix?

Think about the picture we just saw.

setup() once, loop() forever

Command	What it means
<code>pinMode(2, OUTPUT);</code>	“I’m going to control pin 2.”
<code>digitalWrite(2, HIGH);</code>	“Switch the LED on.”
<code>digitalRead(15)</code>	“Is the button pressed?”
<code>analogRead(34)</code>	“How far is the knob turned?” (0–4095)
<code>delay(2000);</code>	“Wait two seconds.”
<code>Serial.println("Hi");</code>	“Show ‘Hi’ in the Serial Monitor.”

Anything after `//` is a **comment**: a note for humans. The computer ignores it.

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *Digital is always more accurate than analog.*

✓ Fact They are types of signal, not levels of quality.

✗ Myth *A pin can power anything directly.*

✓ Fact Fans, motors and pumps need a relay or a driver.

✗ Myth *If the code shows a number, it must be right.*

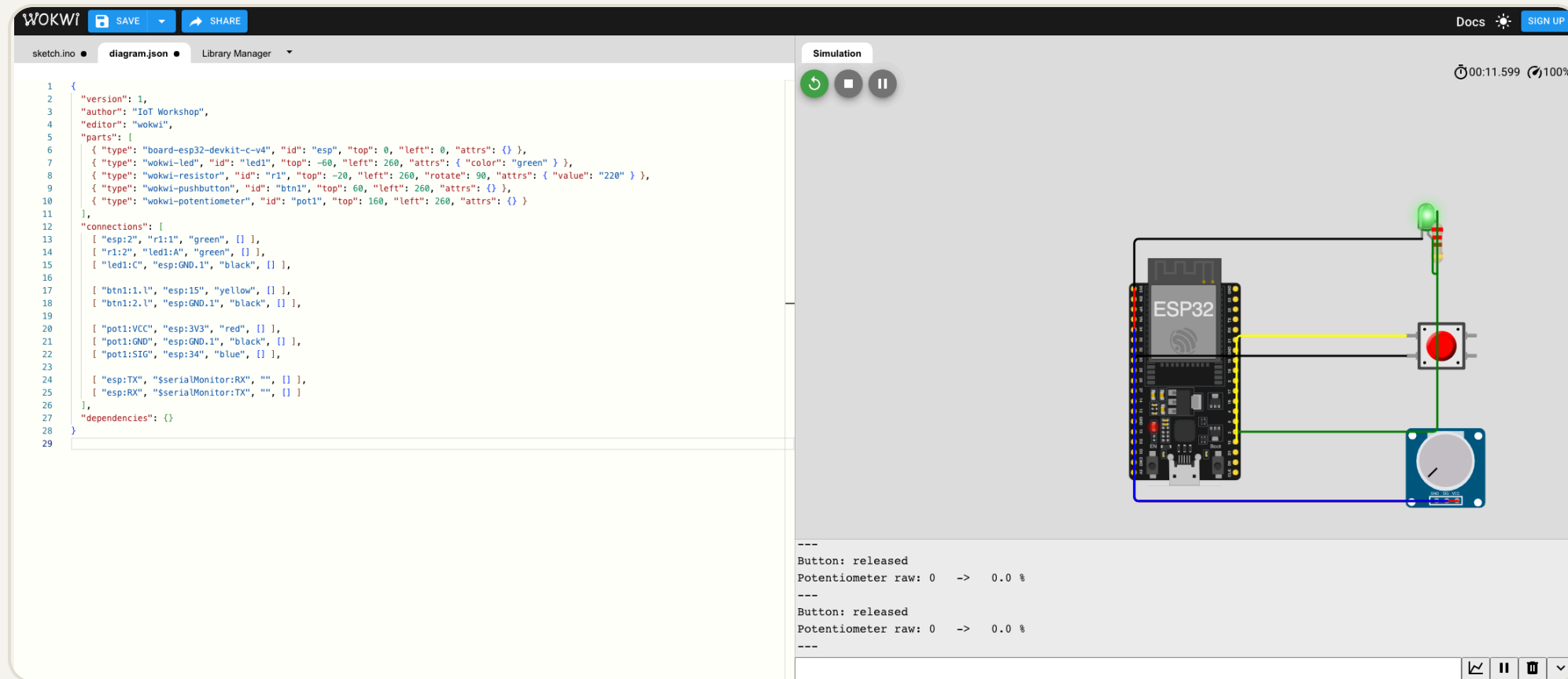
✓ Fact Always ask: is this number sensible?

✗ Myth *A pressed button always reads HIGH.*

✓ Fact With INPUT_PULLUP, a pressed button reads LOW.

LAB 1 · ABOUT 25 MINUTES

Digital vs analog in Wokwi



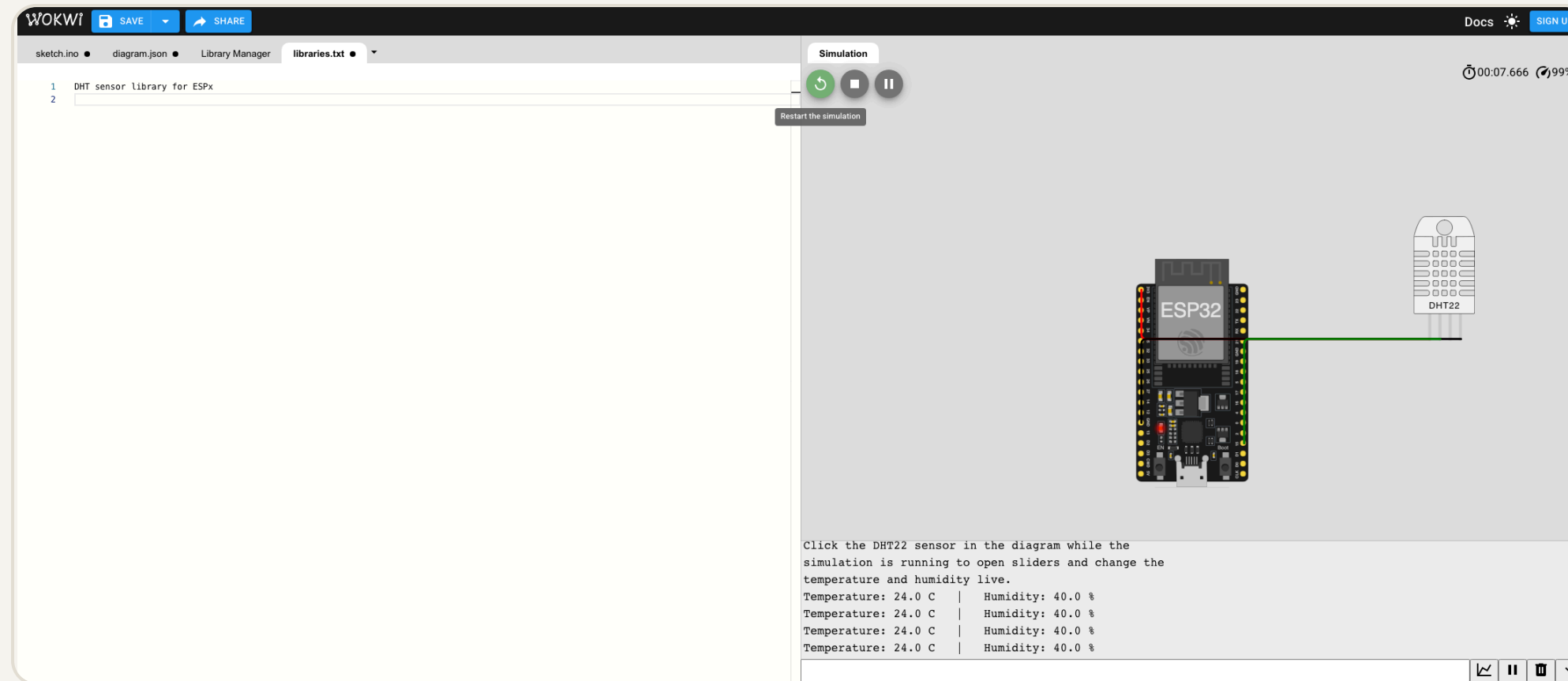
- 1 Open a new ESP32 project in Wokwi
- 2 Paste sketch.ino, then the whole diagram.json
- 3 Press **Play** and watch the LED blink
- 4 Hold the button: the monitor says PRESSED
- 5 Drag the dial: 0 to 4095, shown as a percent



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

LAB 2 • ABOUT 25 MINUTES

Reading a DHT22 sensor



- 1 Open a new ESP32 project, or reuse Lab 1
- 2 Paste sketch.ino and diagram.json
- 3 Add **libraries.txt** with the DHT library name
- 4 Press **Play**; the first build takes longer
- 5 Click the DHT22 and drag its sliders



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

KEY TAKEAWAYS

What to remember

- ✓ **Sensors** turn the physical world into electrical signals
- ✓ **Actuators** act; relays and motors need helper circuits
- ✓ **Digital** = two states. **Analog** = any value in a range
- ✓ The **ADC** gives 0–4095; your code **converts** it to real units
- ✓ **setup()** runs once; **loop()** runs forever

Quick quiz

Q1 Name one sensor with a digital signal and one with an analog signal.

Q2 On an ESP32, `analogRead()` returns 1024. About what percent is that?

Q3 Why can't you connect a fan straight to an ESP32 pin?

Q4 Your code reads a DHT22 ten times a second. Why are the readings strange?

UP NEXT • MODULE 4

Getting Online

Connectivity and communication protocols: Wi-Fi, Bluetooth, LoRaWAN and MQTT — and your DHT22 readings sent across the internet in Lab 3.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

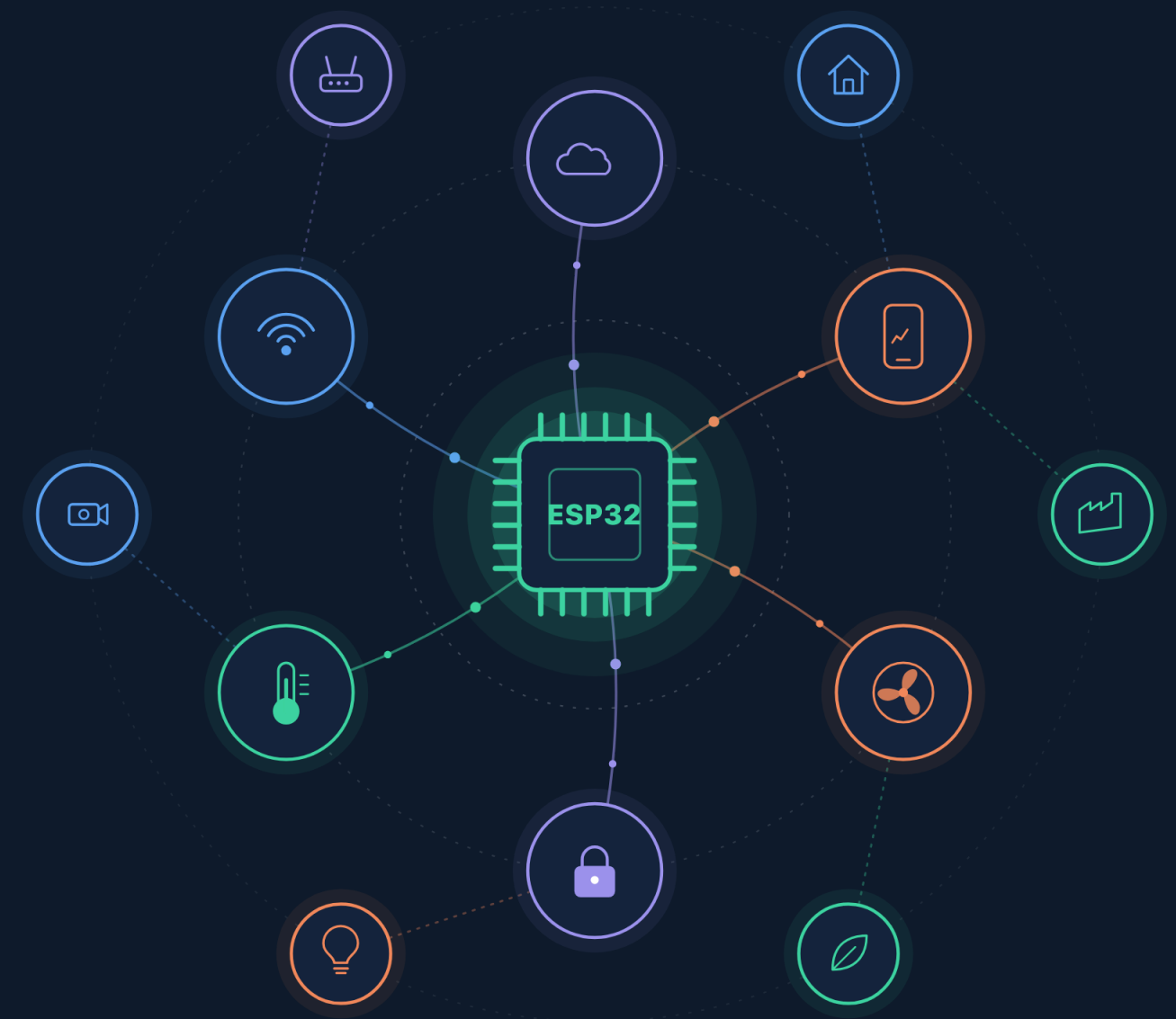
MODULE 4 • 45 MINUTES

Getting Online: Connectivity and Communication Protocols

How devices connect, the “languages” they speak once connected, and how to put your own device online.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

Labs 1–2

04

Getting online

You are here

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Explain** addresses, clients and servers, ports and packets
 - 02 **Compare** Wi-Fi, BLE, Zigbee, LoRaWAN and cellular IoT
 - 03 **Choose** a suitable connection for an IoT project
 - 04 **Explain** how HTTP and MQTT work, and when to use each
 - 05 **Connect** a simulated ESP32 and publish live data over MQTT

Two separate decisions



Protocol

The **language**: MQTT, HTTP/REST, CoAP



Network

Addressing and routing: IP addresses, the internet



Connection

The **road**: Wi-Fi, BLE, Zigbee, LoRaWAN, cellular, Ethernet

In Lab 3: **Wi-Fi** is the road and **MQTT** is the language. In Lab 4: still Wi-Fi, but **HTTP**.

Part A

Networking basics

A handful of words you will keep meeting. You don't need to be a network engineer.

Addresses, names and servers



IP address

Every device's address, like 192.168.1.42



DNS

The phone book: turns names into IP addresses



Router / gateway

Passes data between networks, or radio types



Client / server

The client asks; the server listens and answers

Ports, packets and speed



Port

Which service: 80, 443, MQTT
1883



Packets

Data travels in small chunks



TCP

Reliable, in order: registered post



UDP

Lighter, no promises: ordinary post



Bandwidth

Data per second: the road's width



Latency

How long one message takes to
arrive

PAUSE AND CHECK

Your ESP32 needs to send a reading to `broker.hivemq.com`. What turns that name into an address the network can use?

Part B

Choosing the road

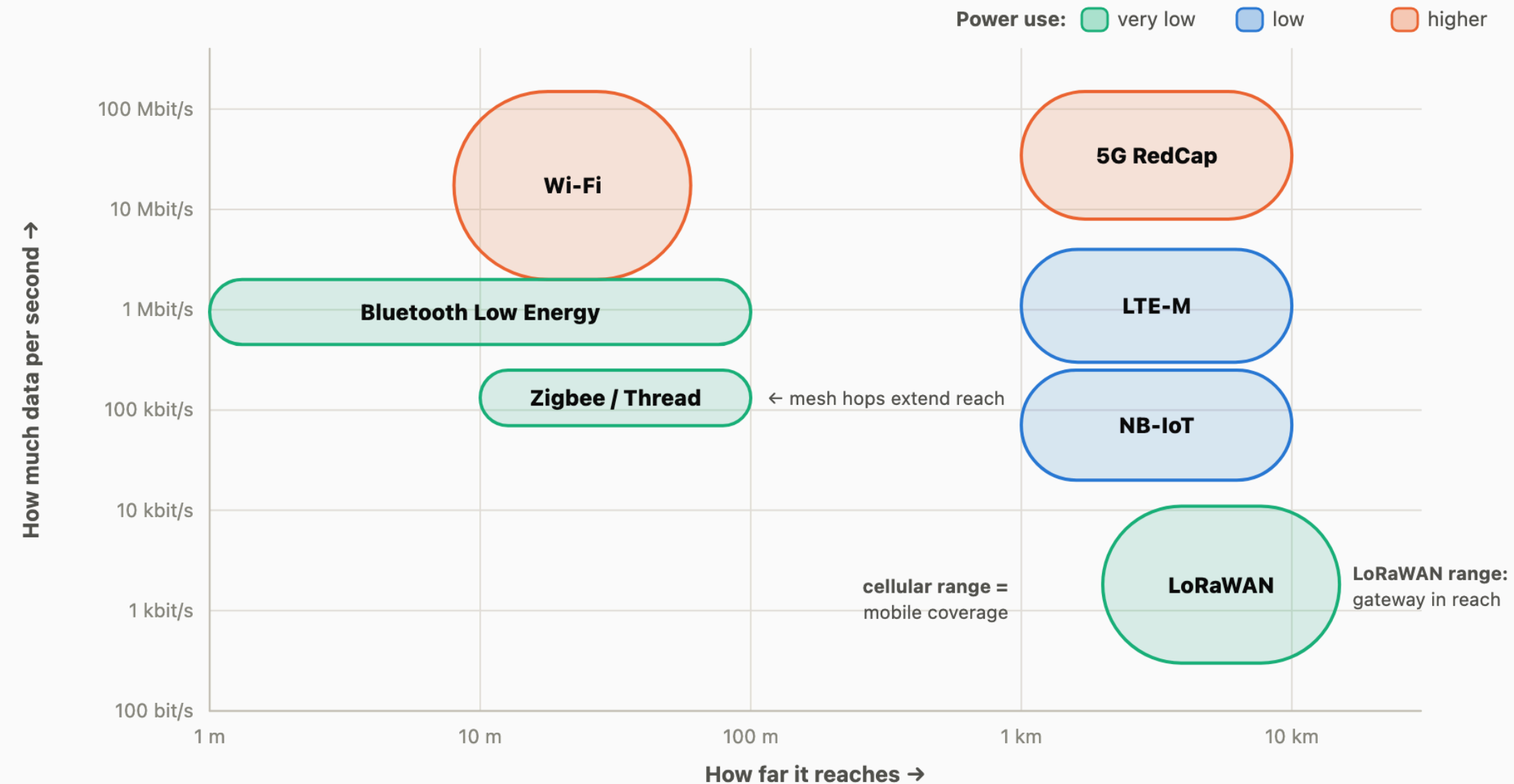
Wi-Fi, Bluetooth, Zigbee, LoRaWAN, mobile networks — and how to pick one.

THE BIG TRADE-OFF

Range, power and data rate

Range vs data rate — and what it costs in power

Typical values, log scales. Colour = power use: the only options with long range AND lots of data use more power.



Wi-Fi: lots of data, but high power

BLE: very low power, short range

LoRaWAN: kilometres, but tiny data

SHORT RANGE

Wi-Fi, Bluetooth LE and Zigbee



Wi-Fi

~30–50 m indoors. Lots of data, high power. Plugged-in devices, cameras.



Bluetooth LE

~10 m. Very low power, years on a coin battery. Wearables, locks.



Zigbee / Thread

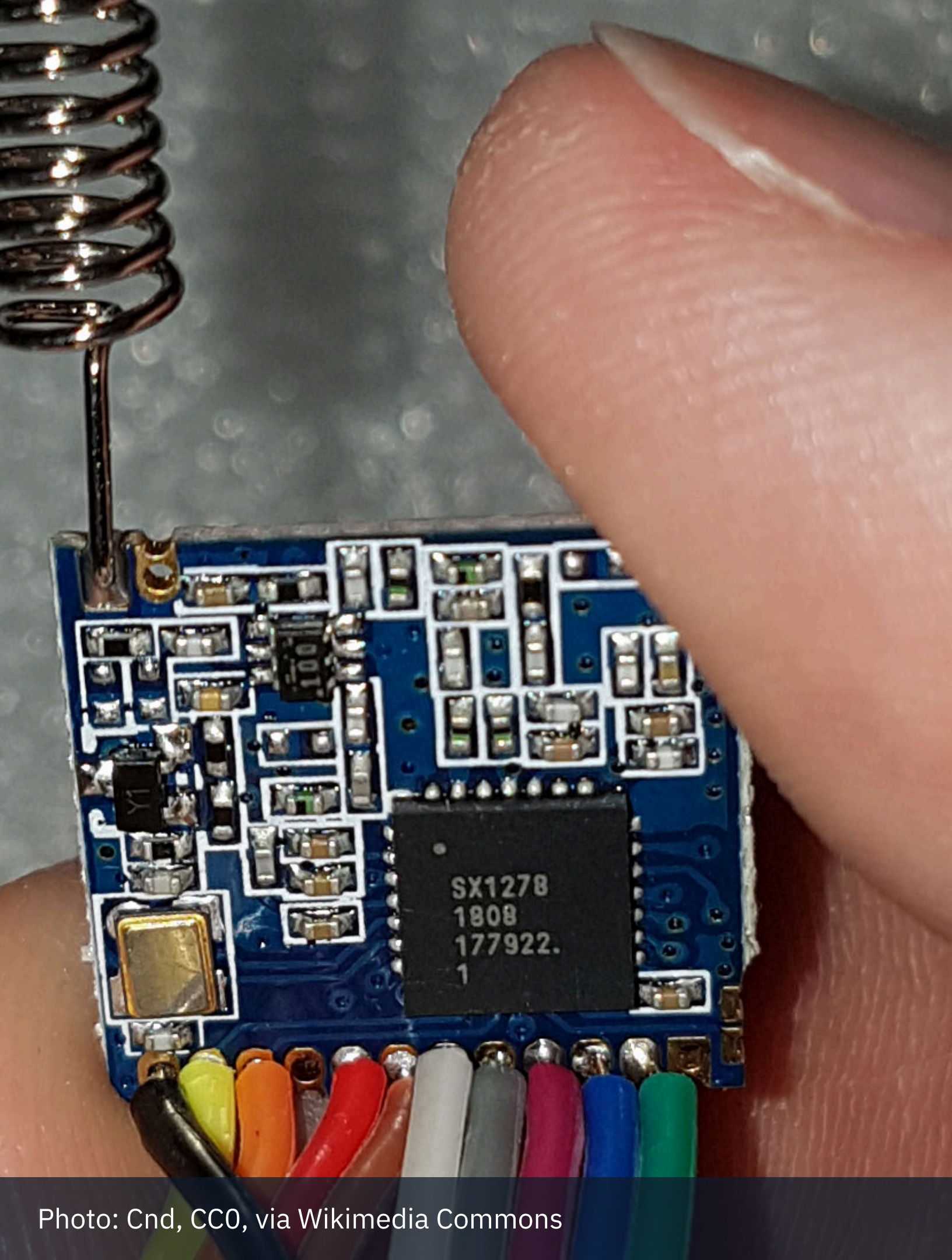
A **mesh**: devices pass messages on. Needs a hub. Smart lights.



LONG RANGE

LoRaWAN

- Often **over 10 km** in the countryside
- A few short messages an hour
- Up to **10 years** on a battery
- Needs a **gateway** within range



INSIDE A LORA SENSOR

A small radio, the right band

- No licence needed for the radio band
- The band differs by country
- Malaysia: **AS923**, about 919–924 MHz

SIDE BY SIDE

Comparing the options

Technology	Range	Power	Needs	Typical use
Wi-Fi	~30–50 m indoors	High	A Wi-Fi router	Cameras, smart plugs
Bluetooth LE	~10 m+	Very low	A phone or gateway	Wearables, locks
Zigbee / Thread	Short, mesh extends	Very low	A hub	Lights, switches
LoRaWAN	Kilometres	Extremely low	A LoRaWAN gateway	Farms, cities
NB-IoT / LTE-M	Mobile coverage	Low	Nothing extra	Meters, trackers
Ethernet	Cable length	Mains	A network switch	Factories

Typical values only. Malaysia switched off 3G in 2022: new cellular products use **NB-IoT, LTE-M or 5G**.

PAUSE AND CHECK

Pick a technology: (a) a river sensor 8 km from town, on a battery; (b) an office security camera; (c) a heart-rate strap that talks to a phone.

Ask: How far? Mains or battery? How much data? What already exists there?

Part C

The languages devices speak

HTTP, MQTT and CoAP — and how to keep the messages safe.

Four methods, a few status codes

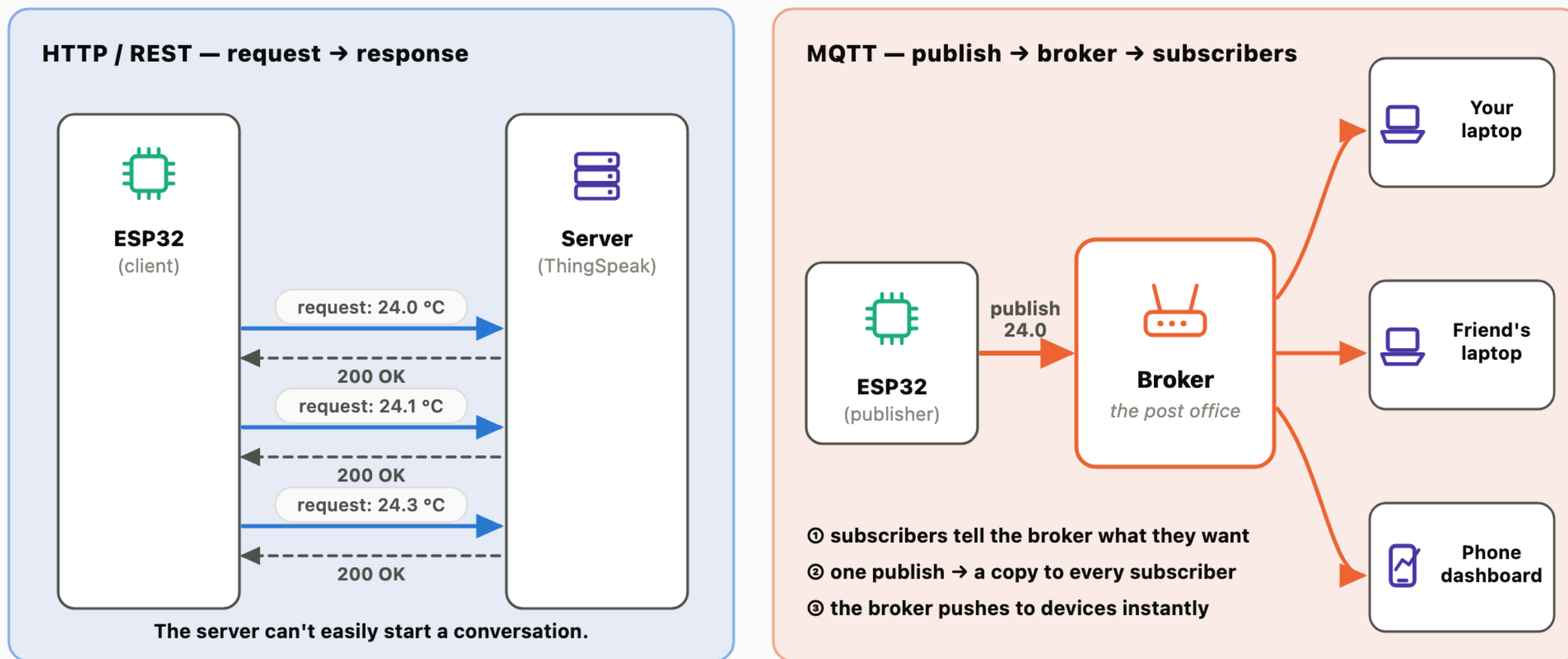
Method	Means	Example
GET	Read something	Get the latest reading
POST	Create or send something new	Send a new reading
PUT	Update something	Change a device’s settings
DELETE	Remove something	Delete a device

Status codes: **200** success · **400** your request was wrong · **404** not found · **500** server problem.

HTTP asks; MQTT posts through a broker

Two ways to talk: ask-and-answer vs the post office

HTTP: the device asks, the server answers — every time. MQTT: publish once, the broker delivers to every subscriber.



Topics and wildcards

Which messages does a subscription receive?

Topics are levels separated by "/". + matches exactly one level. # matches everything below.

Message published to topic...	subscribed to: iot-workshop/+/temperature	subscribed to: iot-workshop/#	subscribed to: iot-workshop/aisha/#
iot-workshop / aisha / temperature	✓	✓	✓
iot-workshop / aisha / humidity	✗	✓	✓
iot-workshop / ravi / temperature	✓	✓	✗
iot-workshop / ravi / humidity	✗	✓	✗
iot-workshop / ravi / room1 / temperature	✗	✓	✗

two levels where + stands → no

CHOOSING A PROTOCOL

HTTP, MQTT or CoAP?

	HTTP/REST	MQTT	CoAP
Pattern	Request–response	Publish–subscribe	Request–response
Runs on	TCP	TCP	UDP
Message size	Larger	Very small	Very small
Server can push?	Not easily	Yes, instantly	Yes, with “observe”
Best for	Occasional uploads	Frequent, live messages	Tiny battery devices
Our labs	Lab 4	Labs 3 and 6	Not used

Rule of thumb: **HTTP** when a device occasionally reports to a web service; **MQTT** for frequent, live messages.

Four risks, four defences



Eavesdropping

Defence: **encryption**
(TLS, HTTPS, MQTT
on port 8883)



Impersonation

Defence:
authentication for
each device



Open topics

Defence: control
who may publish or
subscribe



Leaked secrets

Passwords and API
keys in shared code
can leak

Lab 3 uses a **public** broker on unencrypted port 1883 with no password. For learning only!

REAL-WORLD EXAMPLE

200,000 smart water meters



Connection

NB-IoT or **LoRaWAN**:
basements, 10+ years on a
battery



Protocol

One tiny message a day:
MQTT, CoAP or a compact
format



Security

Encrypted readings; every
meter has its own identity

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *Wi-Fi is always best because it's fastest.*

✓ Fact Wi-Fi drains batteries. In a field, LoRaWAN is far better.

✗ Myth *Bluetooth and Wi-Fi do the same job.*

✓ Fact BLE: low power, short range. Wi-Fi: more data, direct internet.

✗ Myth *MQTT devices send messages to each other.*

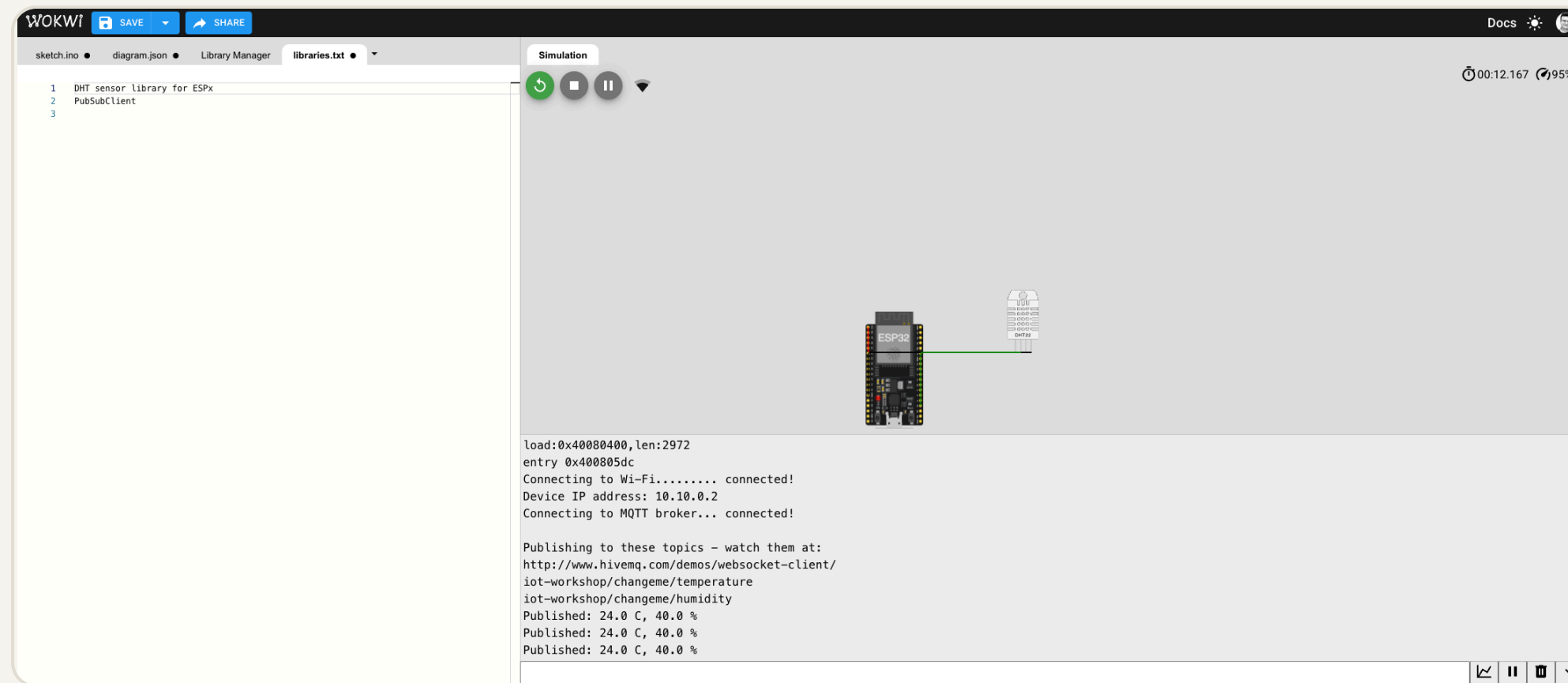
✓ Fact Every message goes through the broker.

✗ Myth *Data on the internet is secure by default.*

✓ Fact Only with encryption and authentication.

LAB 3 • ABOUT 30 MINUTES

Getting online: Wi-Fi + MQTT



- 1 Open a new ESP32 project in Wokwi
- 2 Paste sketch.ino — change “changeme” to your name
- 3 Paste diagram.json and add libraries.txt
- 4 Press Play: Wi-Fi, then MQTT, then readings
- 5 Watch it live in the HiveMQ web client



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

KEY TAKEAWAYS

What to remember

- ✓ Two decisions: **how to connect** (the road) and **what language** to speak
 - ✓ Every radio trades off **range, power and data rate**
 - ✓ Wi-Fi: lots of data · BLE: low power · LoRaWAN: km, tiny data
 - ✓ **HTTP**: ask and answer · **MQTT**: publish–subscribe via a broker
 - ✓ Real systems need **encryption** and **authentication**
-

Quick quiz

Q1 What is the difference between a connection technology and a protocol?

Q2 Farm soil sensors over 5 km, on batteries for years. Which technology — and why not Wi-Fi?

Q3 Does `iot-workshop/+/humidity` match `iot-workshop/lee/room1/humidity`?

Q4 Why is a public broker on port 1883 risky for a real product?

UP NEXT • MODULE 5

IoT Cloud Platforms and Data Visualisation

We give the data a home: what IoT cloud platforms do, and a live graph of your readings in Lab 4 — using HTTP this time.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

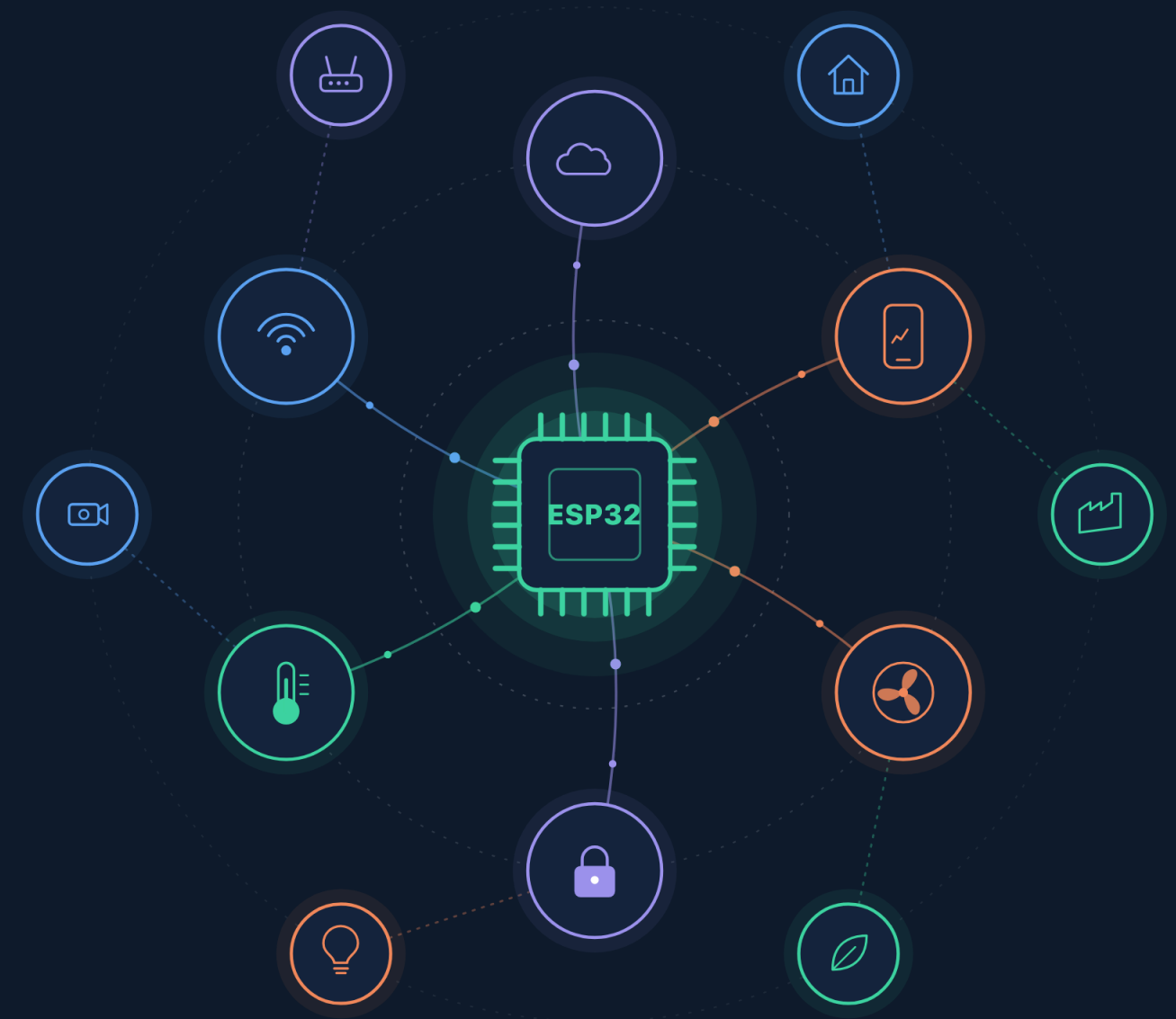
MODULE 5 • 45 MINUTES

IoT Cloud Platforms and Data Visualisation

Where your readings go, how the cloud makes sense of them, and how to show them on a dashboard people understand.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

Labs 1–2

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

You are here

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Explain** what the cloud does for an IoT system
 - 02 **Describe** time-series data and four kinds of analytics
 - 03 **Compare** ThingSpeak, Arduino Cloud, Blynk and AWS IoT Core
 - 04 **Choose** the right chart and design a clear dashboard
 - 05 **Build** a live cloud dashboard in Lab 4
-

Part A

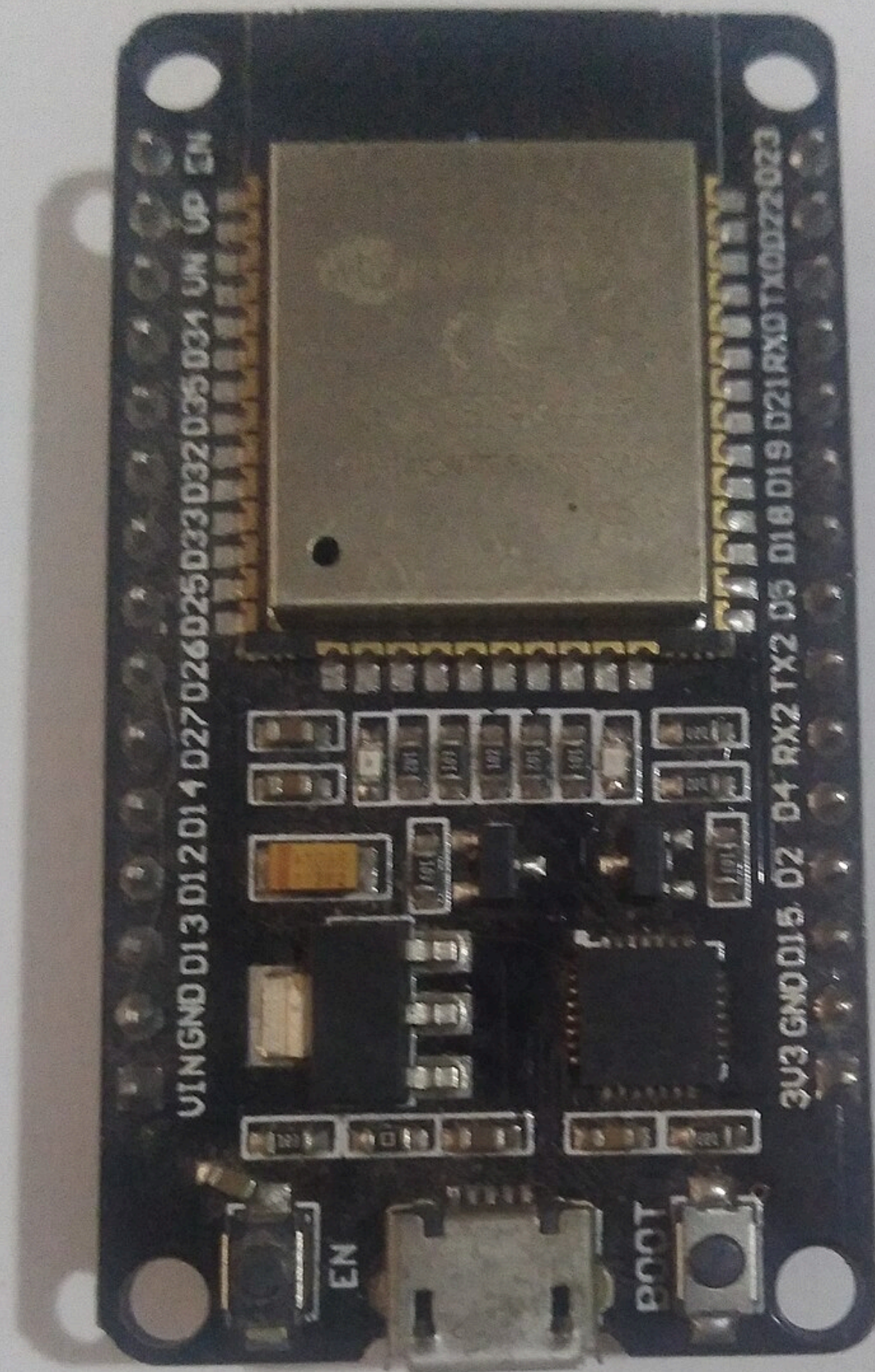
What the cloud does for IoT

In Lab 3, your readings vanished when you closed the tab. Where should they live?

WHAT IS “THE CLOUD”?

Cloud computing means renting computing power, storage and software over the internet.

Like electricity: you don't build a power station to boil a kettle. You plug into the grid and pay for what you use.



WHY IOT NEEDS THE CLOUD

A small board with a tiny memory

An ESP32 is small and cheap on purpose. Could it store years of readings and serve a dashboard to hundreds of people?

What the cloud provides

The cloud provides...	Why it matters for IoT
Storage	Keeps history, so you see trends, not just “right now”
Computing power	Runs analysis far too heavy for a microcontroller
Scale	Handles one device or a million, with no new hardware
Anywhere access	Check your data from any phone or computer
Integration	Connects IoT data to email, billing, maps, AI
Device management	Tracks every device, its status and its permissions

WHAT AN IOT PLATFORM DOES

From device to dashboard



Ingest

Data arrives by MQTT or HTTP.



Manage devices

Credentials, status and settings.



Store

Keep every reading with its time.



Analyse

Averages, trends and alerts.



Visualise

Charts, gauges and dashboards.



Rules and actions

“If this, then that” — in Module 6.

Plus **APIs**: web addresses that let other software read or write your data.

Time-series data: every value has a time



09:00:00

24.0 °C · 40.0 %



09:00:20

24.1 °C · 40.2 %



09:00:40

24.3 °C · 40.1 %

Without the time, 24.3 °C tells you very little. **With** the time, you can see if the room is getting hotter.

HOW MUCH DATA IS THAT?

One sensor, one reading every 20 seconds

4,300

readings a day

1.6 million

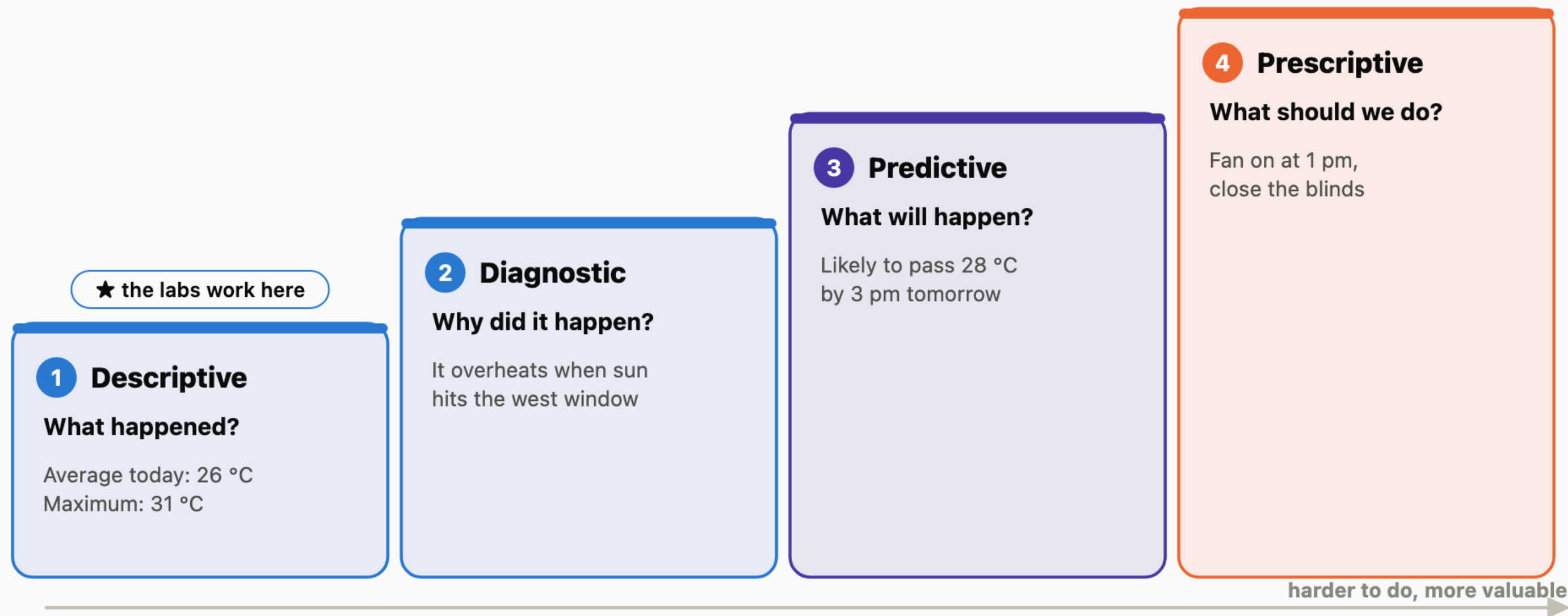
readings a year

Platforms limit **messages** (how many you send) and **retention** (how long data is kept).

Four kinds of analytics

Four levels of analytics

Each step answers a harder — and more valuable — question about the same data.



PAUSE AND CHECK

A farm dashboard shows: “Soil in Field 2 will be too dry in about 6 hours.” Which level of analytics is that?

Descriptive, diagnostic, predictive or prescriptive? Decide with your neighbour.

Part B

Meet the platforms

Four platforms, from the simplest to the most powerful.

ThingSpeak: channels, fields and keys



Channels

One per project or device, like “My Room Monitor”.



Fields

Up to 8 values per channel:
Field 1 = Temperature.



API keys

The **Write** key adds data. The **Read** key reads a private channel.

Free plan: about 3 million messages a year, 4 channels, **15 s** between updates, non-commercial

SIDE BY SIDE

Four IoT platforms compared

Platform	Aimed at	Starting	Strong point	Free option
ThingSpeak	Learners, researchers	Very easy	Simplicity, MATLAB analysis	Yes, non-commercial
Arduino Cloud	Makers, schools	Easy	Code and cloud linked	Yes, very limited
Blynk	App-style prototypes	Easy	Phone control	Yes, limited
AWS IoT Core	Businesses, large scale	Harder	Scale, security, integration	Credits for new accounts

Free-plan details change often. Always check the provider’s pricing page first.

A WORD OF CAUTION

Choosing a platform: four risks



Lock-in

Moving thousands of devices later is painful. MQTT and HTTP help.



Shutdowns

Google Cloud IoT Core shut down in August 2023.



Cost at scale

A free plan for 5 devices says little about 50,000.



Personal data

Data about people must follow laws such as Malaysia's PDPA.

Part C

Dashboards and data visualisation

A good dashboard answers the viewer's questions in seconds.

CHOOSING THE RIGHT CHART

Different questions need different visuals

To show...	Use a...	Example
Change over time	Line chart	Temperature over the last 24 hours
Current value in a range	Gauge or big number	“Tank is 72% full”
A comparison	Bar chart	Energy used by each floor
On/off or good/bad	Status indicator	“Pump: RUNNING”
Where things are	Map	Locations of delivery lorries
Exact values	Table	The last 10 readings, with times

For IoT sensor data, the **line chart** is the workhorse.

Seven rules for a clear dashboard

Rule	In practice
1 Know your audience	A farmer needs “Water Field 3 now”, not raw numbers
2 Most important first	Make it the biggest thing on the screen
3 Always show units	“24” on its own could mean anything
4 Sensible time range	Last hour for “right now”, last week for patterns
5 Show limits	A line for “safe maximum: 28 °C”
6 Not colour alone	Pair red and green with words, like “HIGH” or “OK”
7 Less is more	If nobody acts on a chart, remove it

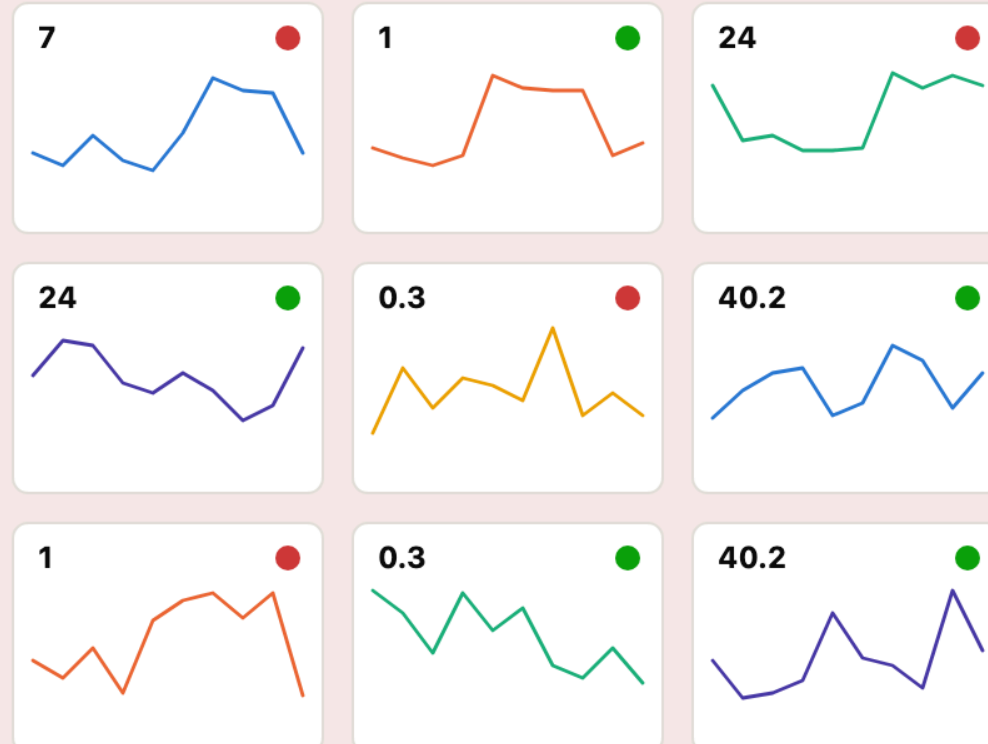
THE SAME DATA, TWO DASHBOARDS

The rules in action

The same data, two dashboards

A good dashboard answers “is everything OK, and what should I do?” in a few seconds.

✗ Hard to read



✗ no units

✗ colour-only status

✗ no clear priority

✗ no limits shown

✓ Easy to read

Living room monitor

2 1 h 24 h 7 d

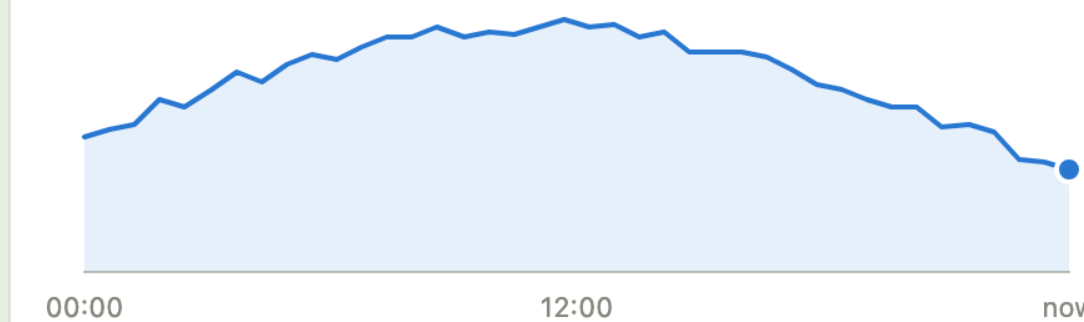
Temperature

Humidity

1 26.4 °C ✓ OK

58 %

3 safe maximum 28 °C



4 ✓ No alerts in the last 24 hours

① number + unit + label ② time range ③ limit shown ④ plain words

PAUSE AND CHECK

A hospital wants one screen showing whether each of 20 medicine fridges is in its safe range. Which visual would you choose, and what would you avoid?

Use the chart table and the seven rules. Discuss in pairs for one minute.

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *The cloud is one big computer somewhere.*

✓ Fact It's many computers in data centres around the world.

✗ Myth *In the cloud, data is automatically safe and permanent.*

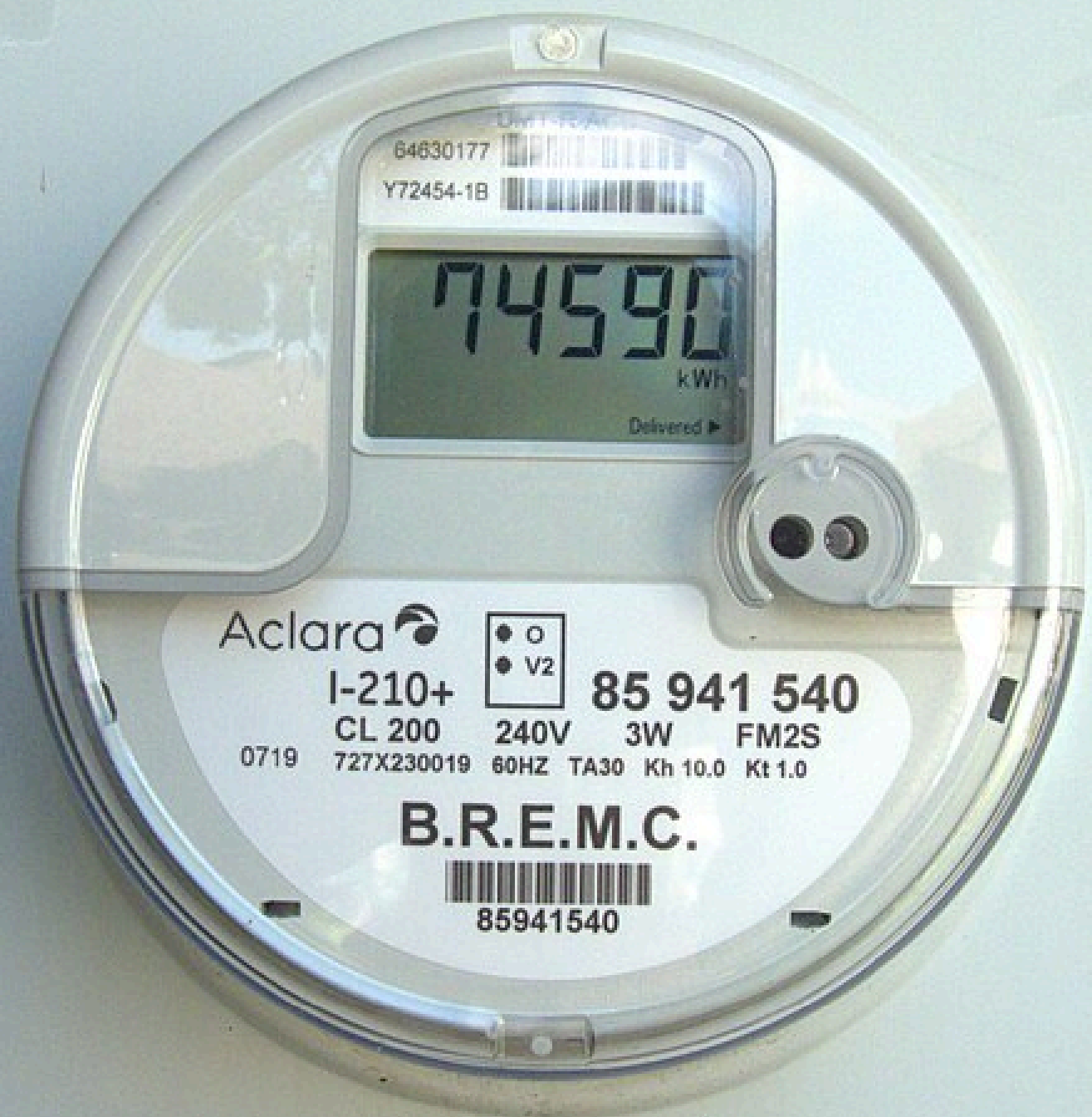
✓ Fact Plans have retention limits, and services can shut down.

✗ Myth *More charts make a better dashboard.*

✓ Fact Usually the opposite. Show a few important things clearly.

✗ Myth *All IoT platforms are basically the same.*

✓ Fact Same core ideas, but very different ease, scale and cost.



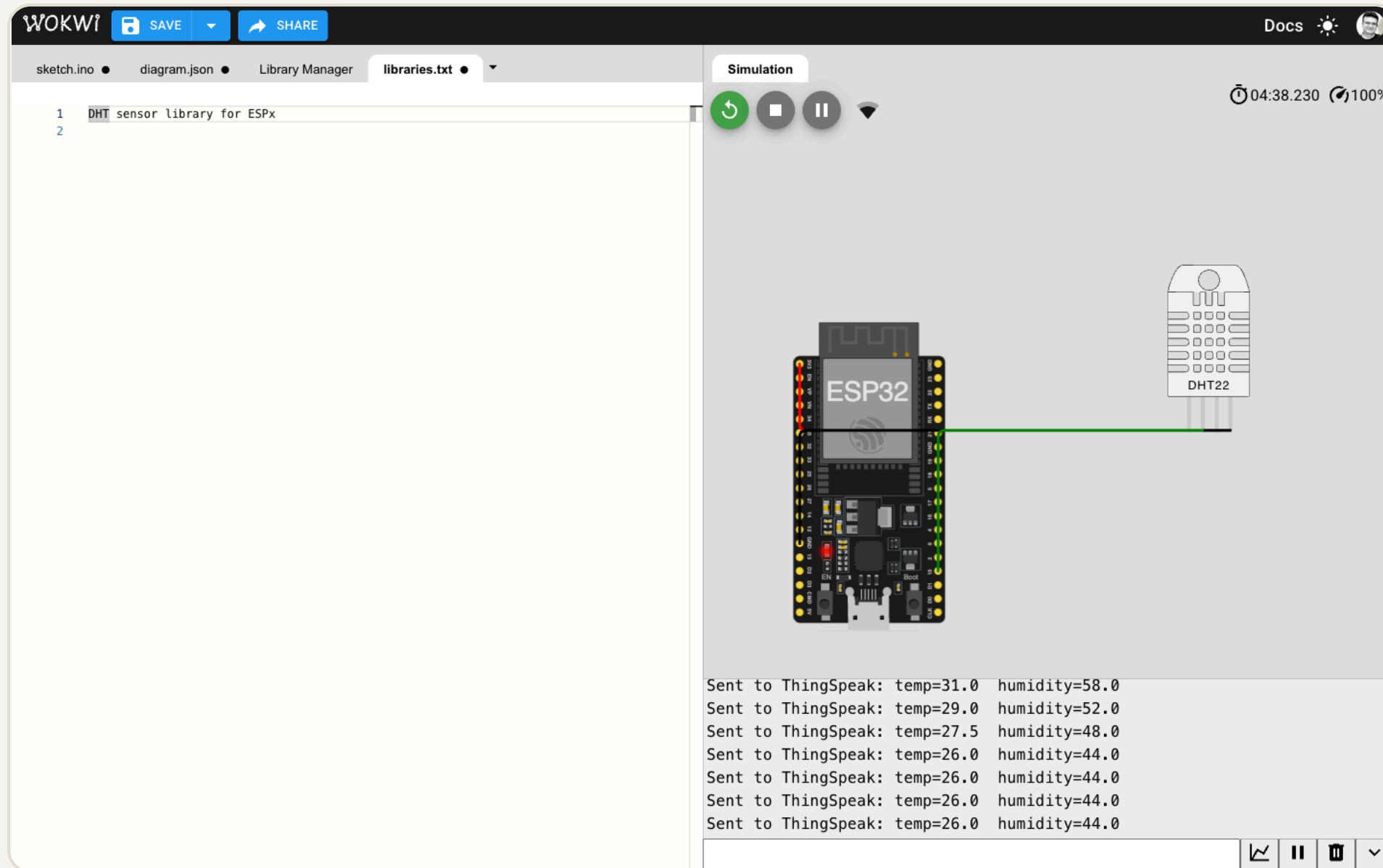
REAL-WORLD EXAMPLE

An energy dashboard for a school

- **Store:** detail for 3 months
- **Analyse:** air-con runs all weekend
- **Principal:** one big number, one line chart
- **Students:** a friendly lobby screen of energy saved

LAB 4 • ABOUT 30 MINUTES

Cloud dashboard with ThingSpeak



- 1 Create a channel with two fields
- 2 Copy your **Write API key**
- 3 Paste the key into the code in Wokwi, then press Play
- 4 Watch your charts grow every 20 seconds



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

KEY TAKEAWAYS

What to remember

- ✓ The **cloud** gives tiny devices storage, computing, scale and access
- ✓ IoT data is **time-series data**: values with timestamps
- ✓ Four levels: **descriptive, diagnostic, predictive, prescriptive**
- ✓ **ThingSpeak, Arduino Cloud, Blynk** and **AWS IoT Core** suit different jobs
- ✓ Good dashboards suit their **audience** and show **units and limits**

CHECK YOUR UNDERSTANDING

Quick quiz

Q1 Name three things the cloud provides that an ESP32 can't.

Q2 What is time-series data, and why is the timestamp so important?

Q3 In ThingSpeak: channel or field? Write key or Read key?

Q4 Which chart shows how a room's humidity changed over the last week?

UP NEXT • MODULE 6

Data Processing and Automation

Your device stops just reporting and starts acting: a relay and a buzzer switch on when it gets too hot — with no cloud needed.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

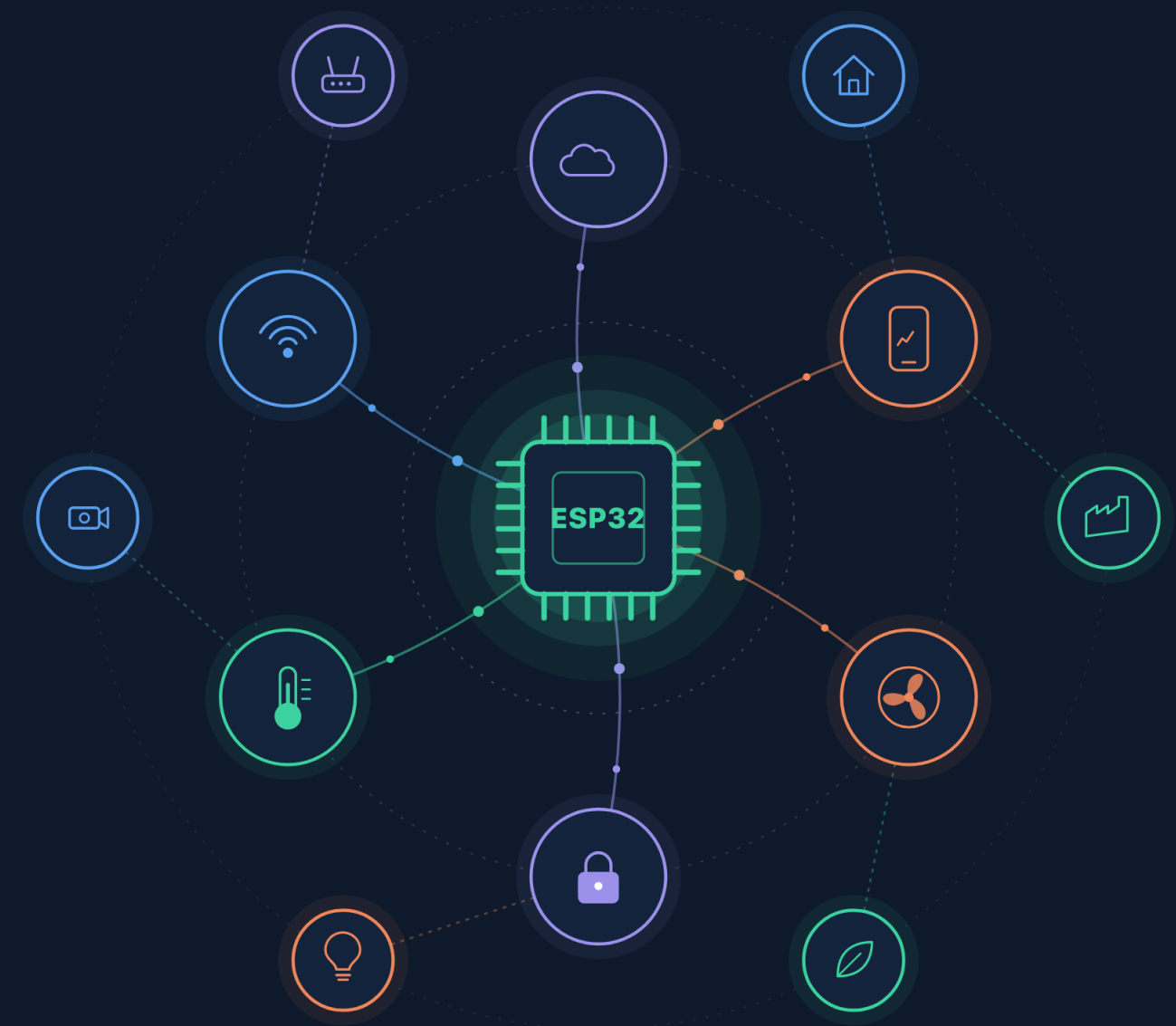
MODULE 6 • 45 MINUTES

Data Processing and Automation

How a device turns raw readings into decisions, and then acts on them by itself.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

Labs 1–2

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

You are here

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Describe** the steps data takes from raw reading to decision
 - 02 **Explain** what “event-driven” means, and why IoT uses it
 - 03 **Write** simple if-then rules, and explain what a rules engine does
 - 04 **Compare** automation at the edge and in the cloud
 - 05 **Build** a device that reacts to a temperature limit by itself
-

WHERE WE ARE

Your device can sense and send. Now it will decide and act, by itself.

A dashboard still needs a person. What if nobody is watching when the server room overheats at 3 am?

THE BIG PICTURE

From reading to action



Raw data

31.4, 31.6, 31.5 °C



Process

Clean, average and
compare



Rule

Is it above 28 °C?



Act or wait

Yes: fan on + alert.
No: keep watching.

Three ideas: **processing, events and rules**, and **where the decision happens**.

Part A

From raw data to useful information

Raw sensor data is rarely ready to use as it is.

Seven steps from reading to decision

Step	What happens	Room monitor example
1. Collect	Read the sensor at a sensible rate	Read the DHT22 every 2 seconds
2. Validate	Throw away readings that can't be real	Ignore -40 °C or "not a number"
3. Clean / smooth	Reduce noise and random wobble	Average the last 5 readings
4. Transform	Convert units or combine values	"Feels like" from temperature + humidity
5. Aggregate	Summarise over time	Hourly average, daily maximum
6. Decide	Compare against rules	Is the average above 28 °C?
7. Store / act / send	Save it, act on it, or pass it on	Switch the fan, log it, send an alert

Not every system uses every step. But almost all do some version of **collect, check, clean, decide**.

One bad reading can cause a false alarm



Validate

A loose wire sends **85 °C**. Rule: ignore any reading more than 10 °C away from the last one.



Smooth

Near the limit, readings jump: 27.9, 28.1, 27.8, 28.2. Average a few before you decide.

PAUSE AND CHECK

Your sensor reports 24.1, 24.3, 24.2, 97.0, 24.2 °C. Which processing step should deal with the 97.0?

Look back at the seven steps. Think on your own for 30 seconds.

Part B

Events and rules

Noticing when something important happens, and deciding what to do about it.

Two ways to handle data



Polling

Check again and again:
“Temperature now? And
now?”



Event-driven

React when something
meaningful happens.



An event

A change that matters:
crossed 28 °C, door opened.

A child asking “Are we there yet?” every five minutes is polling. A driver saying “We’ve arrived!” is an event.

Why IoT systems are event-driven



Saves resources

One message when something changes, not 1,000 “nothing changed”.



Fast

React the moment it happens, not at the next check.



Scales

One event, many reactions: fan, alert, log, dashboard.

This is how **MQTT** works (Module 4): publish once, and every subscriber reacts.

AUTOMATION RULES

IF this, THEN that

IF (condition)	THEN (action)
Temperature > 28 °C	Switch the fan on
Motion detected AND after 10 pm	Switch on the porch light
Soil moisture < 20%	Open the irrigation valve for 5 minutes
No data for 15 minutes	Email the technician

Combine conditions with **AND** and **OR**. The service **IFTTT** is named after this pattern: “If This Then That”.

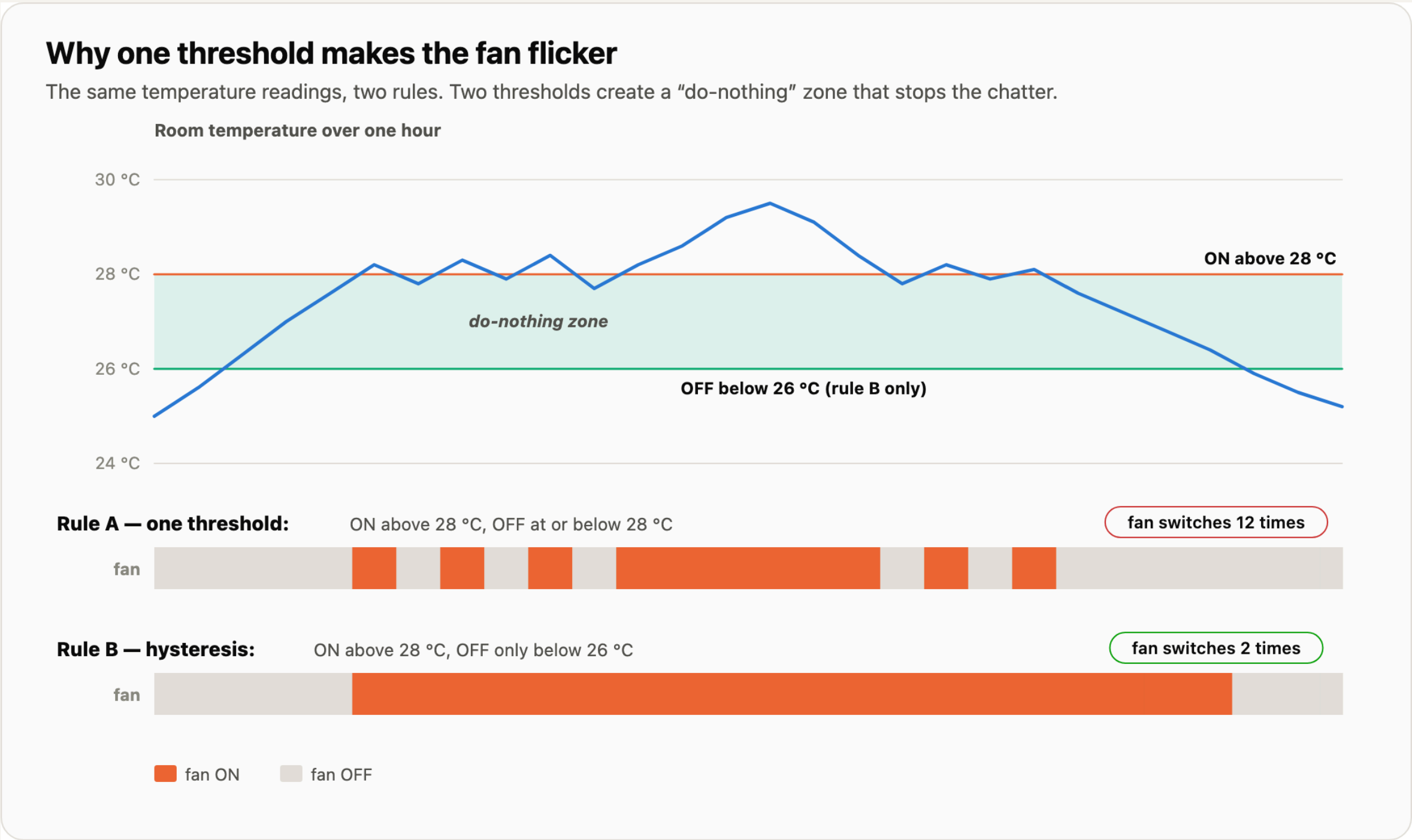
Rules engines in IoT platforms

Platform	Its rule feature, in plain terms
ThingSpeak	“React”: run an action when a channel’s value meets a condition
Blynk	“Automations”: if-then rules built in the app
AWS IoT Core	“Rules”: route and act on device messages as they arrive
Node-RED	Free, visual: build rules by connecting boxes with wires
Home Assistant	Free smart-home hub with automations for home devices

The idea is always the same as the “if” in Lab 5. Only where it runs and how you edit it change.

A CLASSIC RULE PROBLEM

Flickering, and the fix: hysteresis



PAUSE AND CHECK

Write a rule in plain English: alert if a freezer rises above -10°C , but only if it stays that warm for more than 10 minutes.

Use IF ... AND ... THEN. Why the 10 minutes? Talk to your neighbour for one minute.

Part C

Where should the decision happen?

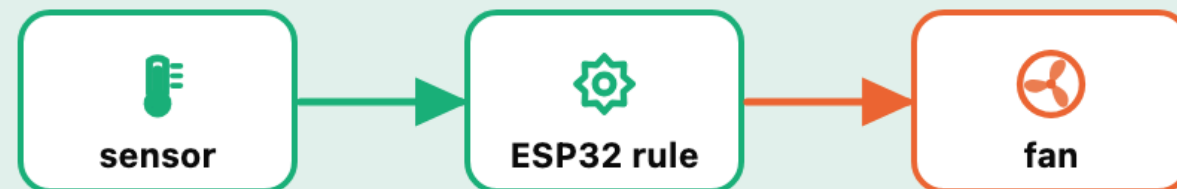
On the device, at the edge — or far away, in the cloud.

The same rule, two places to run it

The same rule, two places to run it

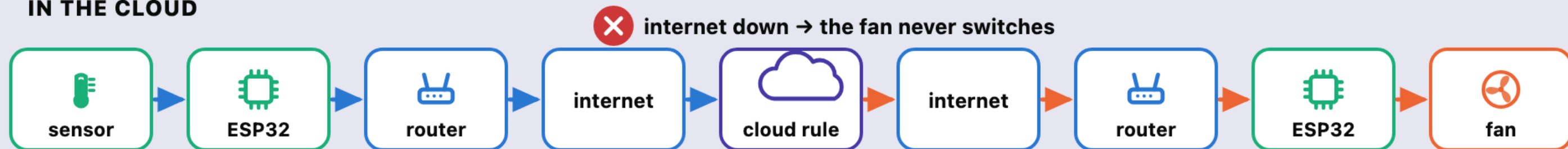
At the edge the decision takes two short hops. In the cloud it's a round trip — and it breaks if the internet does.

AT THE EDGE



✓ **≈ milliseconds · 2 hops**
keeps working with the internet down

IN THE CLOUD



≈ hundreds of milliseconds to seconds · 8 hops

but: easy to change for every device at once, and it sees data from all devices

Blue arrows: reading going up · orange arrows: command coming back

PROS AND CONS

Edge automation and cloud automation

	At the edge	In the cloud
Speed	Fast: a fraction of a second	Slower: a round trip over the internet
Internet down	Keeps working	Stops: no internet, no decision
Changing a rule	Update every device’s software	Edit once, applies to every device
What it sees	Only itself	Data from all devices
Computing power	Limited: a microcontroller	Lots, for heavy analysis and history
Data and cost	Sends less, saves money	Ongoing costs for all that data
Privacy	Sensitive data can stay on the device	More data sent, more privacy risk

CHOOSING, AND USING BOTH

Where should this rule run?

If the decision...	Put it...
Is urgent or safety-critical	At the edge
Must keep working without internet	At the edge
Needs data from many devices	In the cloud
Changes often, or is set by non-technical staff	In the cloud
Needs heavy analysis or long history	In the cloud

Most good systems use **both**: urgent, simple rules at the edge; big-picture rules in the cloud.

REAL-WORLD EXAMPLE

A cold-chain vaccine fridge



Processing

Validate, then average over five minutes.



Events

Left the safe range, power failed, offline.



Edge rules

A local buzzer at once, even with no internet.



Cloud rules

Text the pharmacist, escalate, keep a record.

Some vaccines must stay between **2 °C and 8 °C** at all times, even while they travel.

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *Automation means artificial intelligence.*

✓ Fact Most automation is simple if-then rules. AI builds on top.

✗ Myth *Edge and cloud compete. You must pick one.*

✓ Fact They are partners. Most systems split the work.

✗ Myth *Sending every reading to the cloud is safest.*

✓ Fact It costs more, drains batteries and fails offline.

✗ Myth *One threshold is enough for a good rule.*

✓ Fact It flickers. Use hysteresis or a time delay.

NEW PART FOR LAB 5

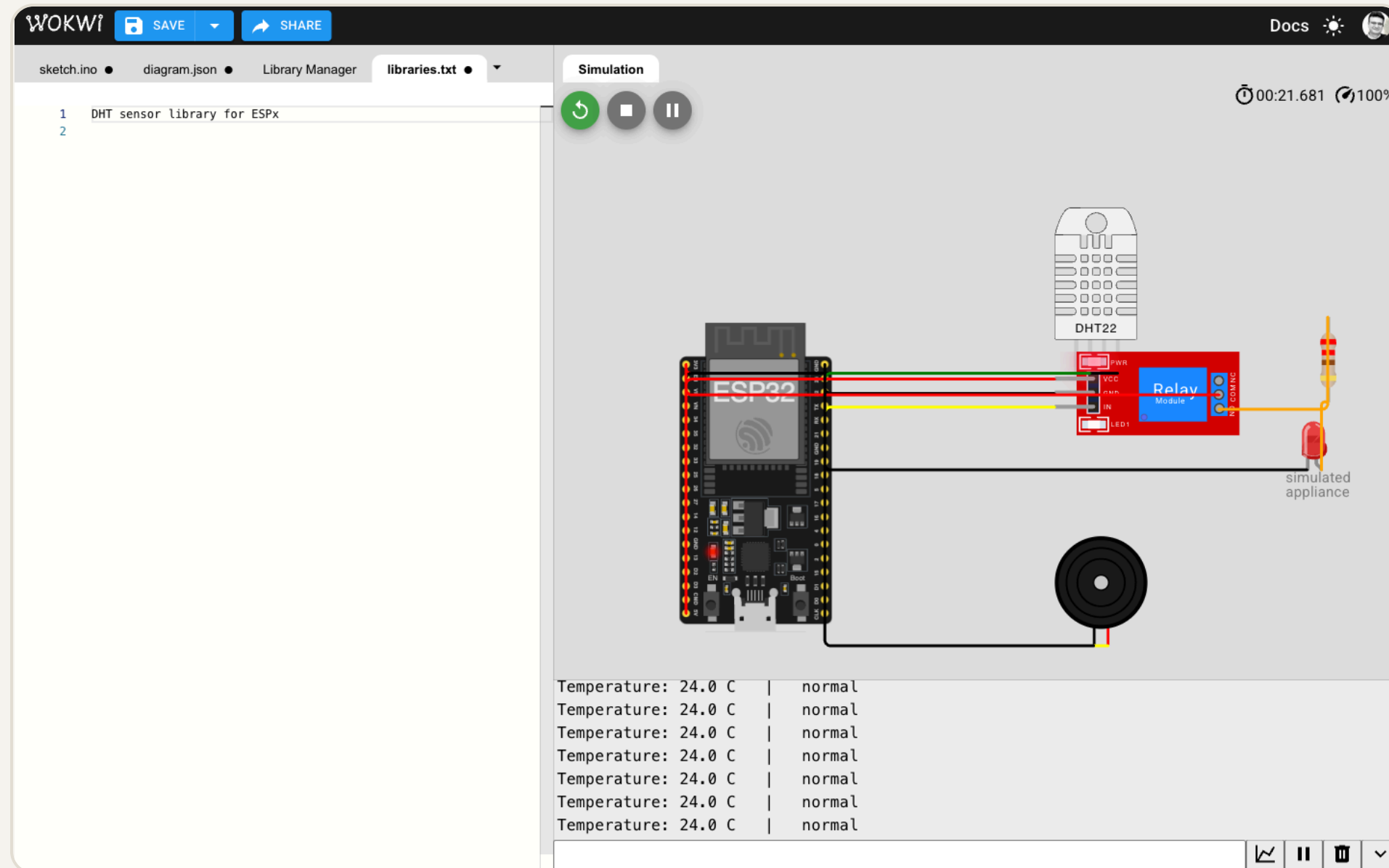
The relay module

- An **electrically controlled switch**
- Switches a much bigger circuit on or off
- The ESP32 drives it like an LED: one pin
- In Lab 5, an **LED stands in** for the fan



LAB 5 • ABOUT 30 MINUTES

Automation: threshold-triggered relay



- 1 Open a new ESP32 project in Wokwi
- 2 Paste sketch.ino, diagram.json and libraries.txt
- 3 Press Play: a reading every 2 s, all “normal”
- 4 Click the DHT22 and drag the temperature above 28 °C
- 5 Watch the relay, LED and buzzer switch on by themselves



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

KEY TAKEAWAYS

What to remember

- ✔ Data flows through a **workflow**: collect, validate, clean, decide, act
- ✔ An **event** is a meaningful change; event-driven saves power and data
- ✔ Rules follow **IF** condition **THEN** action, with AND / OR
- ✔ A **rules engine** runs many rules you can change without code
- ✔ **Hysteresis** (on at 28 °C, off at 26 °C) stops flickering
- ✔ **Edge** is fast and works offline; **cloud** sees the big picture

Quick quiz

Q1 What's the difference between a reading and an event? Give an example of each.

Q2 A greenhouse fan switches on and off every few seconds. What's the problem, and the fix?

Q3 A gas-leak detector must shut a valve instantly, even with no internet. Edge or cloud?

Q4 Why would a large company use a rules engine instead of rules in device code?

UP NEXT • MODULE 7

IoT Security and Best Practices

We step back from building and look at what can go wrong: how IoT devices get attacked, and how real products defend against them.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

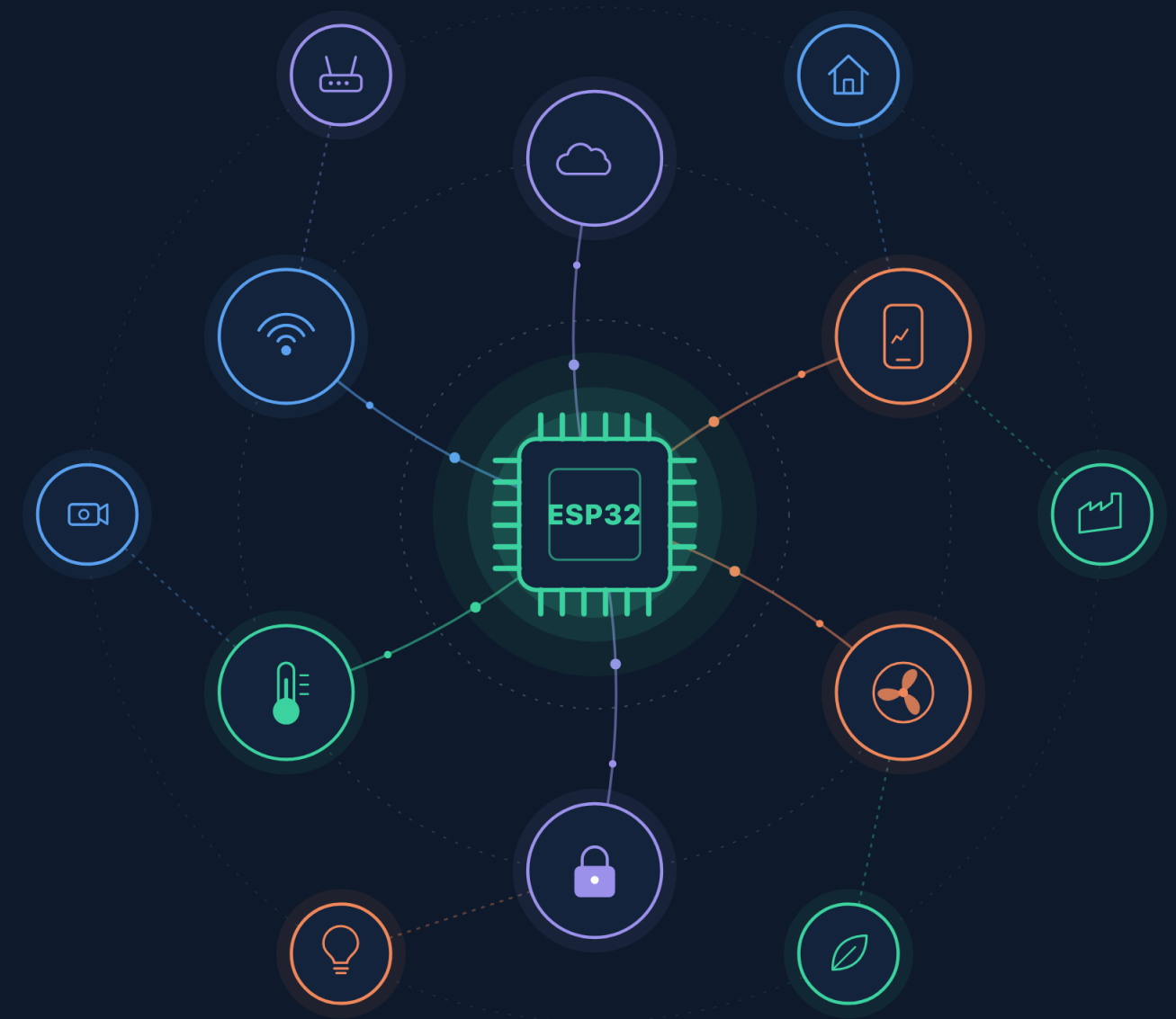
MODULE 7 • 45 MINUTES

IoT Security and Best Practices

How IoT devices get attacked, the proven defences, and the standards and laws that now apply.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

Labs 1–2

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

You are here

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Explain** why IoT is a target, and its most common weaknesses
 - 02 **Explain** authentication, authorisation and encryption
 - 03 **Describe** what makes a firmware update secure
 - 04 **Name** the main standards and laws, including Malaysia's
 - 05 **Spot** common security mistakes in code and settings
-

WHERE WE ARE

The shortcuts we took in the labs



Public broker

Lab 3: a public MQTT broker, with no password and no encryption.



Plain HTTP

Lab 4: data sent over plain HTTP, with the API key in the code.



Wi-Fi in code

In all the labs, the Wi-Fi details are written into the code.

Fine for simulated temperature readings. In a real product, each one is a **security hole**.

Part A

Why IoT is a target

What attackers look for, and what we need to protect.

Why IoT devices are attractive targets



So many

Billions of devices,
and many are
identical.



Rarely updated

A weakness found in
year one is still
there in year five.



Rushed design

Weak passwords,
old software, and no
way to fix them.



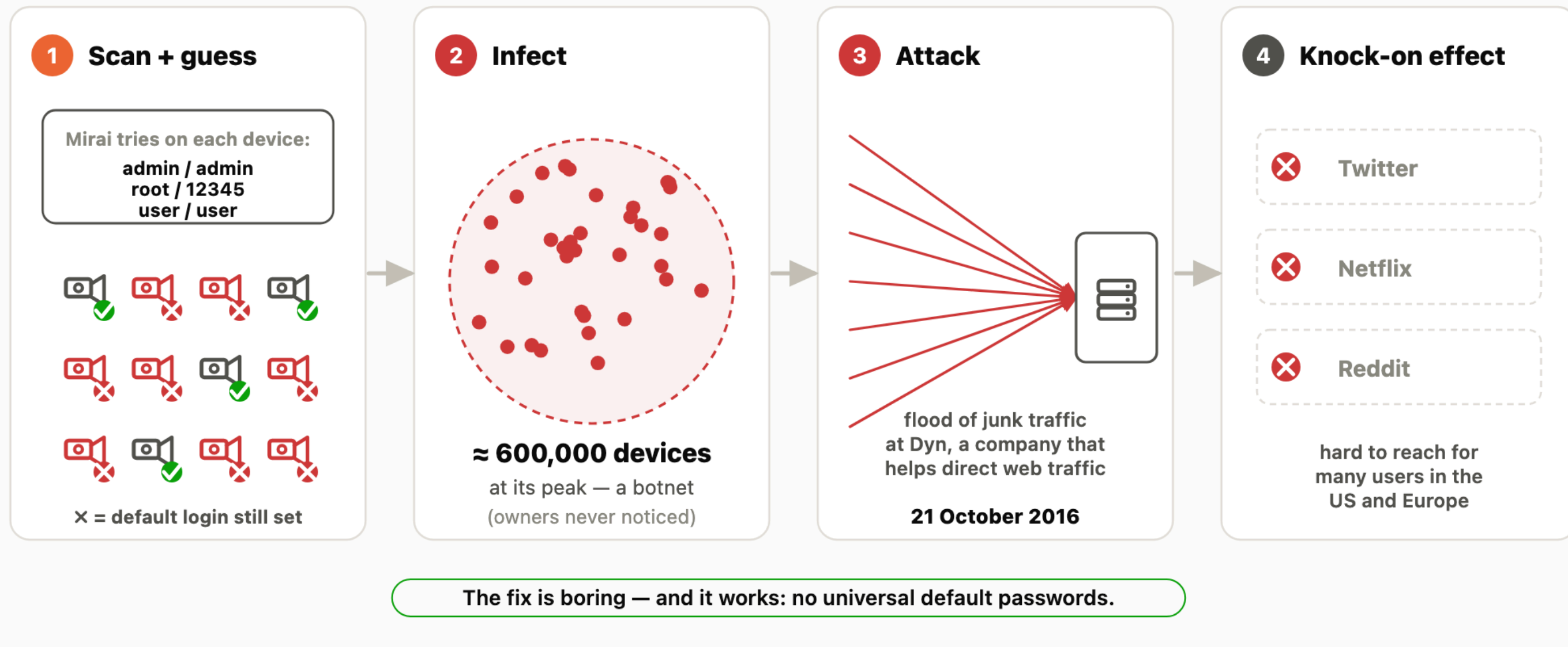
Physical world

An attack can
unlock a door, or
switch off a freezer.

The Mirai botnet

How default passwords built the Mirai botnet (2016)

No clever hacking — just a short list of factory logins, tried on every device it could find.



WHAT ARE WE PROTECTING?

The CIA triad

Goal	Means	When it goes wrong
Confidentiality	Only the right people can see the data	A stranger watches your baby-monitor camera
Integrity	Data can't be secretly changed	A fake sensor reading shuts down a factory
Availability	It keeps working when needed	A smart lock won't open: its cloud is down

In IoT there is often a fourth goal: **safety**. A security failure can put people in danger.

The attack surface



Devices

Default passwords, open ports,
physical tampering



Connectivity

Eavesdropping and fake messages



Edge

Local tampering and gateways that
are never patched



Cloud platform

Weak access control and stolen
API keys



Application

Web dashboards and phone apps
that are insecure



People

Phishing and unchanged settings

THE MOST COMMON WEAKNESSES

The OWASP IoT Top 10

#	Weakness	#	Weakness
1	Weak, guessable or hardcoded passwords	6	Insufficient privacy protection
2	Insecure network services	7	Insecure data transfer and storage
3	Insecure ecosystem interfaces	8	Lack of device management
4	Lack of secure update mechanism	9	Insecure default settings
5	Insecure or outdated components	10	Lack of physical hardening

Published in 2018 and still current. Most items are about **defaults** and **neglect**.

PAUSE AND CHECK

Think back to our lab shortcuts: a public broker, no encryption, and secrets in the code. Which OWASP items do they match?

Look at the Top 10 again. Talk to your neighbour for one minute.

Part B

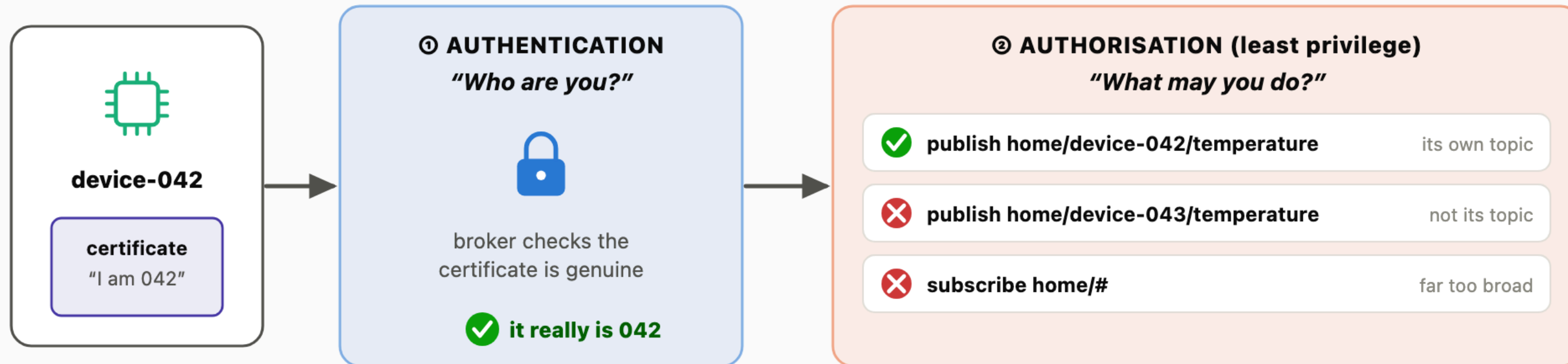
Four defences

Strong identity, encryption, secure updates, and secure by design.

Authentication and authorisation

Authentication vs authorisation

Two separate checks at the MQTT broker: who are you? — then — what are you allowed to do?



Everyday comparison: a hotel

● **Passport at check-in**
= authentication (who you are)

🔒 **Key card opens only YOUR room**
= authorisation (what you may do)

FROM WEAKEST TO STRONGEST

How devices prove who they are

Method	How it works	Strength
No authentication	Anyone can connect, like the Lab 3 broker	None — learning only
Shared password	Every device uses the same password	Weak: steal one, get all
Unique key per device	Each device has its own secret	Better: one leak, one device
Certificate per device	A unique certificate and a secret key	Strongest: professional

With certificates and encryption, **both sides** check each other: *mutual authentication*.

Protect data in transit and at rest

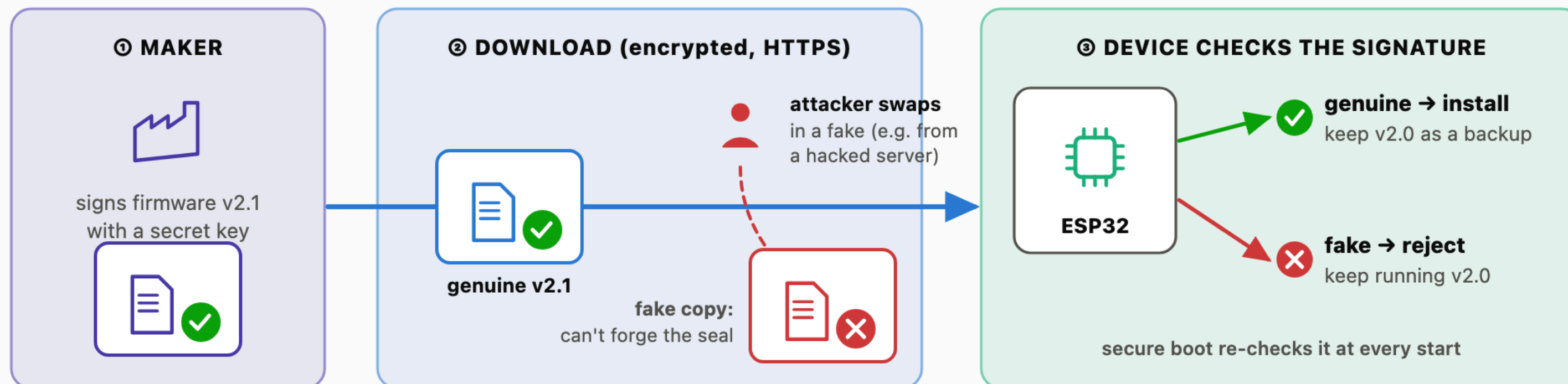
	Insecure	Secure
Web requests	http:// (port 80)	https:// (port 443)
MQTT	port 1883	port 8883, with TLS
Stored data	Plain text	Encrypted

Keep secrets out of code. And use **both** encryption and authentication: TLS checks the server’s certificate.

A secure firmware update

A secure firmware update, step by step

The device installs an update only if the maker's digital signature checks out.



Why each step matters

- Signing proves WHO made it and that nobody changed it.
- Encryption stops others reading or tampering with it on the way.
- Keeping the old version means a failed update never "bricks" the device.

DEFENCE 4

Secure by design



Minimise

Switch off every port and service you don't need.



Defence in depth

If one layer fails, another still holds.



Collect less

Data you never collect can never be leaked.



Whole life

Plan how it's set up, updated and retired.



Easy to report

Let researchers report weaknesses to you.

Part C

Standards and laws

From “nice to have” to “required by law”.

Four standards to know

Name	From	What it is
OWASP IoT Top 10	OWASP, a non-profit	The ten most common weaknesses: a checklist
ETSI EN 303 645	The European standards body	Consumer IoT baseline; no default passwords
NIST IR 8259	The US standards agency	Guidance for device makers; Rev. 1 in April 2026
IEC 62443	International Electrotechnical Commission	For industrial systems: zones and security levels

IoT security laws



UK PSTI Act

Since April 2024: bans guessable default passwords.



EU CRA

Full rules from 11 December 2027.



US Cyber Trust

A voluntary label, still rolling out.



Cyber Security Act

Act 854, for critical sectors; overseen by NACSA.



PDPA (2024)

Personal data breaches reported within 72 hours.



MCMC codes

Technical codes on IoT security.

A summary, not legal advice. For a real product, check the latest official sources.

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *My device is too small and boring to be attacked.*

✓ Fact Tools like Mirai scan the whole internet for any weak device.

✗ Myth *Hiding the Wi-Fi network name makes it secure.*

✓ Fact Hidden networks are easy to find. Use passwords and encryption.

✗ Myth *Encryption solves everything.*

✓ Fact It doesn't stop weak passwords, fake updates or stolen keys.

✗ Myth *Once it's secure, it stays secure.*

✓ Fact New weaknesses appear all the time. Devices need updates.

REAL-WORLD EXAMPLE

Securing our smart room monitor

In the labs	In a real product
Public MQTT broker, anyone can join	Private broker, or a platform like AWS IoT Core
No authentication	A unique certificate for each device
Port 1883, unencrypted	Port 8883 with TLS encryption
Any client can read <code>iot-workshop/#</code>	Each device may only use its own topics
Plain HTTP to ThingSpeak	HTTPS
Wi-Fi password and API key in the code	Secrets stored separately, never shared
No way to update the device	Signed OTA updates, stated support period
—	A published way to report security problems

Spot the vulnerability

- 1 Form groups of 2–3. Each group takes 2–3 of the five snippets.
- 2 For each snippet, ask: **what's wrong?**
- 3 Then ask: **what's the realistic worst case?**
- 4 And: **how would you fix it?** Every fix is in this module.
- 5 Share your answers: one sentence each.

DEBRIEF

Five snippets, five fixes



Wi-Fi on GitHub

Store secrets in a separate file that's never shared.



admin / admin

Force a new, unique password at first setup.



Plain HTTP

Use HTTPS, and MQTT on 8883 with TLS.



One lock topic

Give each lock its own identity and its own topic.



Unsigned update

Sign it, send it over HTTPS, and check the signature.

Every one of these is on the **OWASP IoT Top 10** checklist.

KEY TAKEAWAYS

What to remember

- ✓ Mirai showed the damage **default passwords** alone can do
- ✓ Protect **confidentiality, integrity, availability** — and **safety**
- ✓ **Authentication** proves who; **authorisation** limits what
- ✓ Encrypt **in transit** and **at rest**; keep **secrets out of code**
- ✓ Updates must be **signed, encrypted** and able to **fall back**

Quick quiz

Q1 Why did the Mirai botnet take over so many devices?

Q2 Authentication or authorisation? Give an IoT example of each.

Q3 What is data in transit and data at rest? How do you protect each?

Q4 A smart camera recognises residents' faces. Which Malaysian law applies?

UP NEXT • MODULE 8

Real-World IoT Applications and Project Showcase

We pull the whole day together: complete systems from real industries, from prototype to deployment, and your capstone in Lab 6.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop

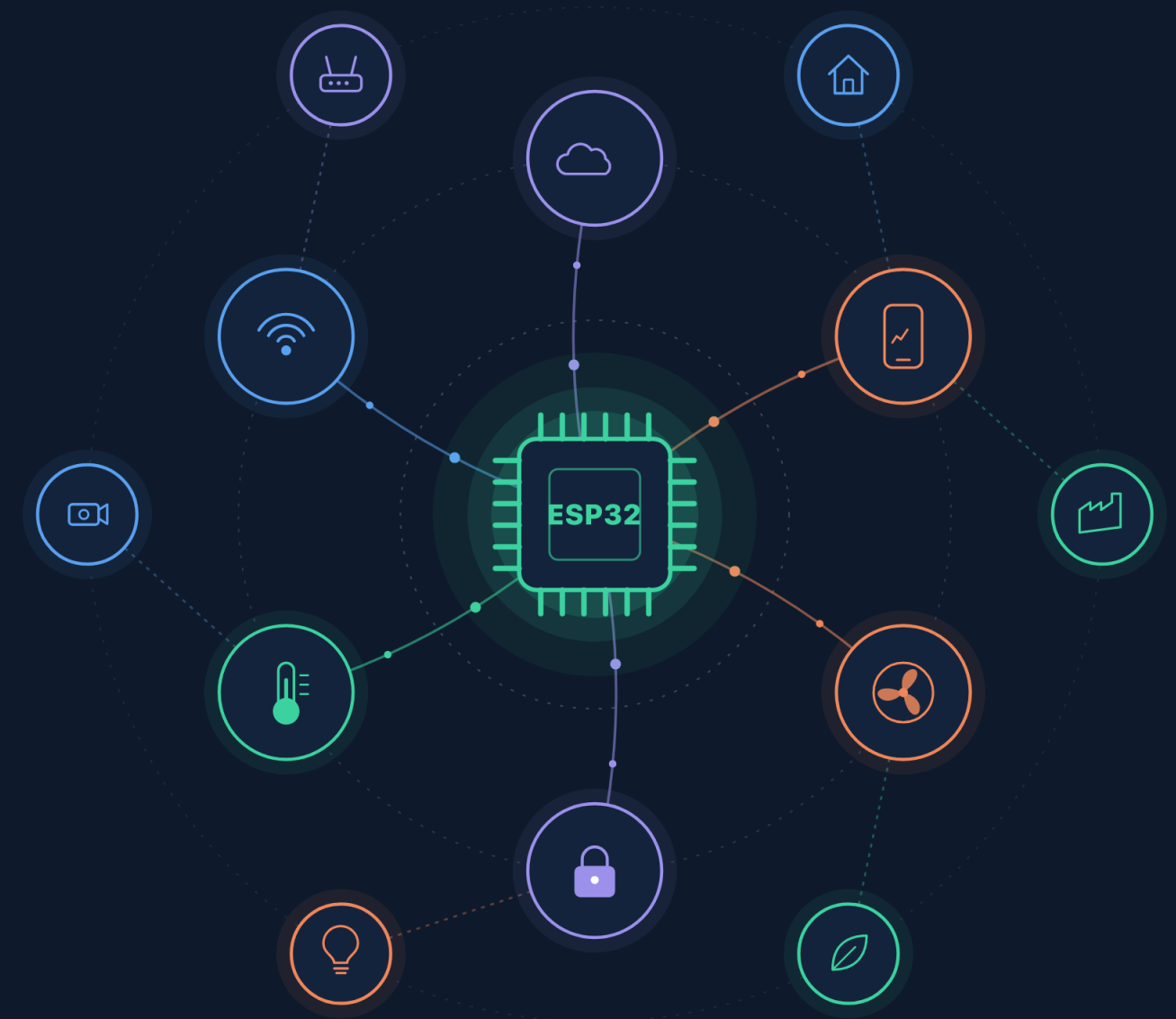
MODULE 8 • 45 MINUTES

Real-World IoT Applications and Project Showcase

How professionals design real IoT systems, how real industries use them, and your own capstone project.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD SO FAR

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

Labs 1–2

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

You are here

BY THE END OF THIS MODULE

You will be able to...

-
- 01 **Follow** a step-by-step method to design an IoT system

 - 02 **Explain** the design choices in homes, industry and farms

 - 03 **Describe** how teams prototype an idea quickly and cheaply

 - 04 **Identify** the main challenges of deploying IoT for real

 - 05 **Build and present** a complete IoT project

THE BIG PICTURE

From problem to deployed system



1. Problem

What problem, and for whom?



2. Requirements

What must it do, and where?



3. Design

Choose each of the five layers.



4. Prototype

Simulate first, then real hardware.



5. Pilot

A small test in the real world.



6. Deploy

Run it — then learn and improve.

Lessons from the real world loop back to the requirements for the next version.

Part A

A design method you can reuse

Turning the five layers into a series of questions.

STEP 1 · THE PROBLEM AND THE PEOPLE

“Our greenhouse loses about 10% of its seedlings each summer because nobody notices when it overheats at weekends.”

Ask: what problem? Who uses it? What is it worth?

Eight questions to ask

Question	Why it matters
What must we measure or control?	Chooses the sensors and actuators
How often, and how accurately?	Battery life, data volume, sensor choice
Where will the devices be?	Connectivity, enclosures and power
How are they powered?	Often the hardest constraint of all
How fast must it react?	What runs at the edge or in the cloud
How many devices, now and later?	Cost, management and platform choice
What’s the budget?	Rules options in or out quickly
What data is personal or sensitive?	Shapes security and privacy design

STEP 3 • DESIGN EACH LAYER

Choose each layer — and write down why



Devices

Which sensors, actuators and chip?



Connectivity

Which radio? Which protocol?



Edge

What must be decided locally?



Cloud platform

Where is data kept and analysed?



Application

Who sees what, on which screen?



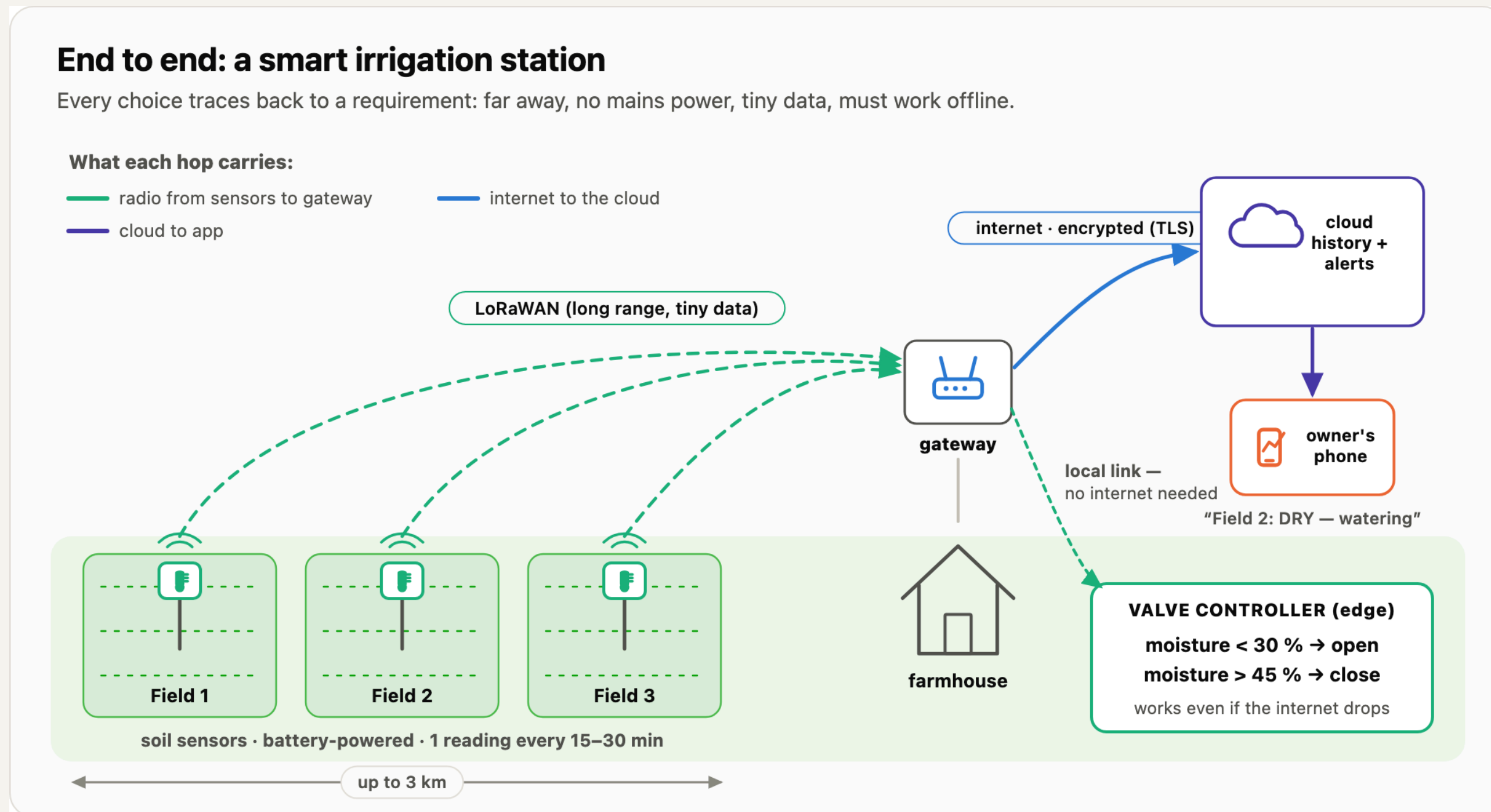
Security

Applies across every layer.

Every choice traces back to a requirement

Layer	Choice	Why
Devices	Soil and temperature sensors; relay for the valve	Measures what matters; switches the valve safely
Connectivity	LoRaWAN to one farmhouse gateway	Kilometres of range, years on a battery
Edge	Open below 30% moisture, close above 45%	Works if the internet drops; no flickering
Cloud	Store readings; alert if a sensor is silent for 2 h	Checks across all fields, plus history
Application	Phone dashboard: one status tile per field	See what needs attention in seconds
Security	Own keys per sensor; MFA; signed updates	Hard to fake readings; can be patched

The smart irrigation station



PAUSE AND CHECK

Why is the “open the valve” rule at the edge, while the “sensor has gone silent” rule is in the cloud?

Think about what happens when the internet is down. Talk to your neighbour for one minute.

Part B

Case studies from real industries

Same five layers every time. Watch why each choice suits its environment.

CASE STUDY 1

Smart home automation



Connectivity

Wi-Fi for cameras;
Zigbee or Thread for
battery sensors.



Matter

Devices from
different brands
work together.



Edge

A local hub keeps
rules running offline.



Challenges

Privacy, easy setup,
many brands.



CASE STUDY 2

Industrial equipment monitoring

- Vibration, heat and current sensors on motors
- **Edge gateways** analyse data on site
- Cloud predicts failures across the factory
- Downtime cut by **30–50%** (McKinsey)
- Old machines, harsh sites, strict security



CASE STUDY 3

Smart agriculture

- Soil, weather and tank-level sensors
- **LoRaWAN** or **NB-IoT**: big farms, small messages
- Irrigation decided locally; cloud adds forecasts
- Challenges: power, weather, rural signal, cost

WHAT THE CASE STUDIES HAVE IN COMMON

Same five layers, different choices

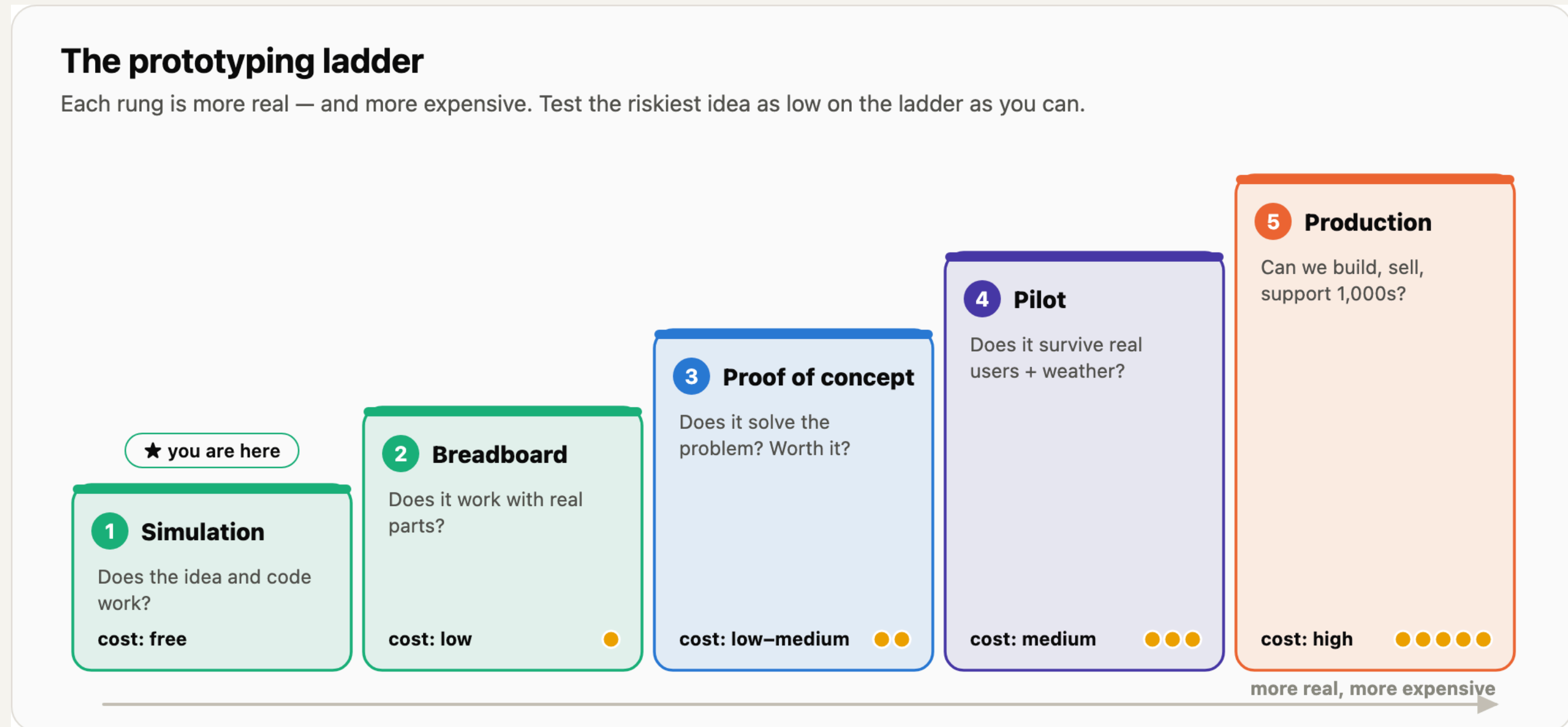
	Smart home	Industry	Agriculture
Main goal	Comfort, security, energy	Avoid breakdowns	Save water and labour
Connectivity	Wi-Fi, Zigbee/Thread, BLE	Wired + gateways	LoRaWAN, NB-IoT
At the edge	Local hub rules	Vibration analysis	Irrigation control
In the cloud	Remote access, history	Cross-factory prediction	Weather-aware planning
Biggest challenge	Privacy, ease of use	Old machines, safety	Power, remote places

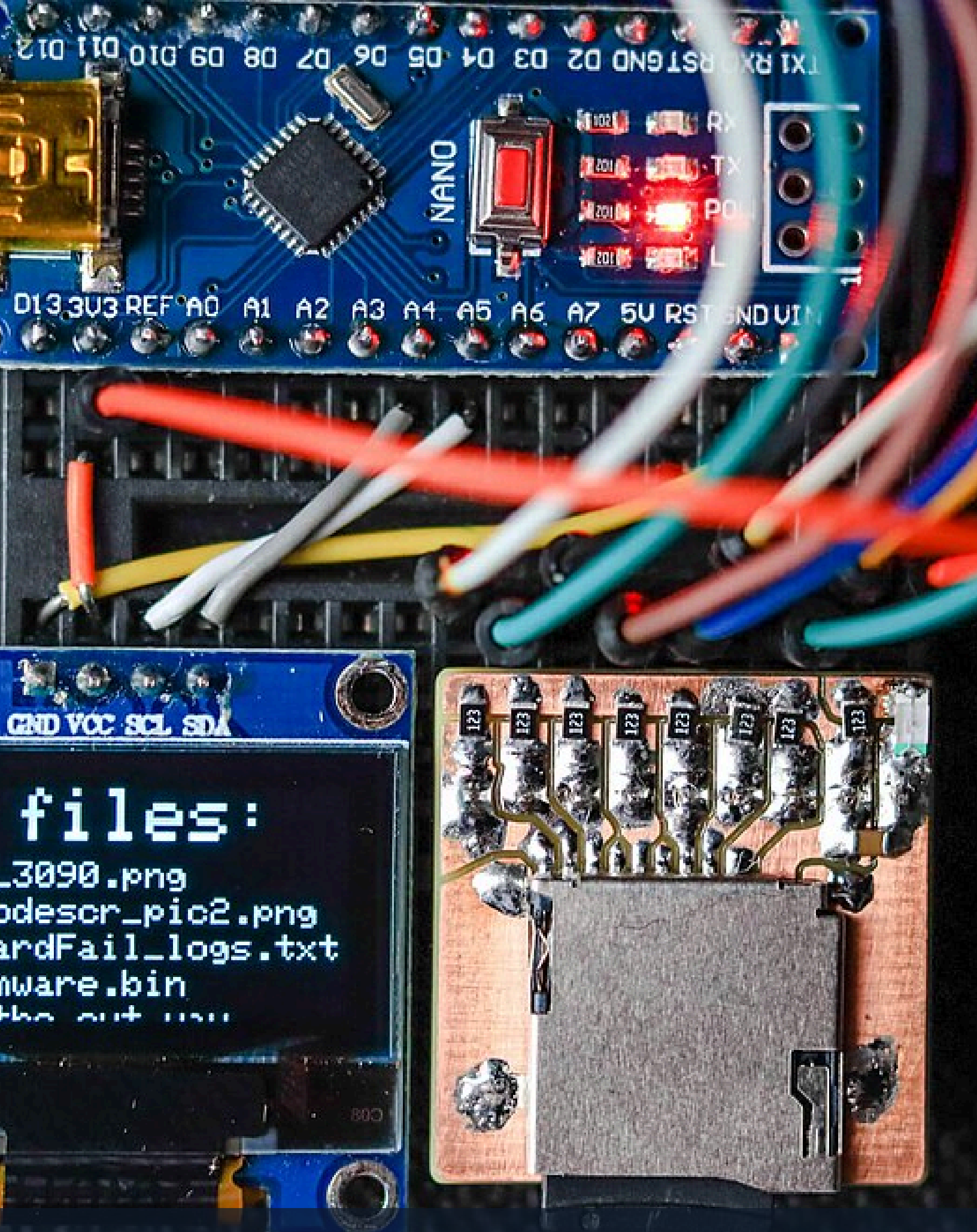
Part C

Prototyping, then the real world

Test ideas cheaply first. Then face the problems the lab never shows you.

The prototyping ladder





PROOF OF CONCEPT

What a proof of concept must prove

- **Can it be done?** Range, battery, accuracy
- **Is it worth doing?** Savings, and real users
- Works but saves nothing? Still a failed PoC
- Test your **riskiest idea** first

Challenges out in the field



Power

Deep sleep;
measure real
battery life.



Connectivity

Store readings
offline, then send
them later.



Environment

Heat and humidity:
use a strong case.



Data quality

Sensors drift:
calibrate them.

Challenges of running at scale



Scale

Set up and update thousands.



Standards

Open standards avoid lock-in.



Total cost

Fees, visits and batteries, for years.



Security & law

Certified radios; laws like PDPA.



People

Involve and train users early.

Memory aid: **power, signal, scale, weather, data, standards, cost, security, people.**

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *If the prototype works, the product is almost done.*

✓ Fact A prototype is often less than half the journey.

✗ Myth *Design the perfect system before building anything.*

✓ Fact Real teams learn by building cheap prototypes.

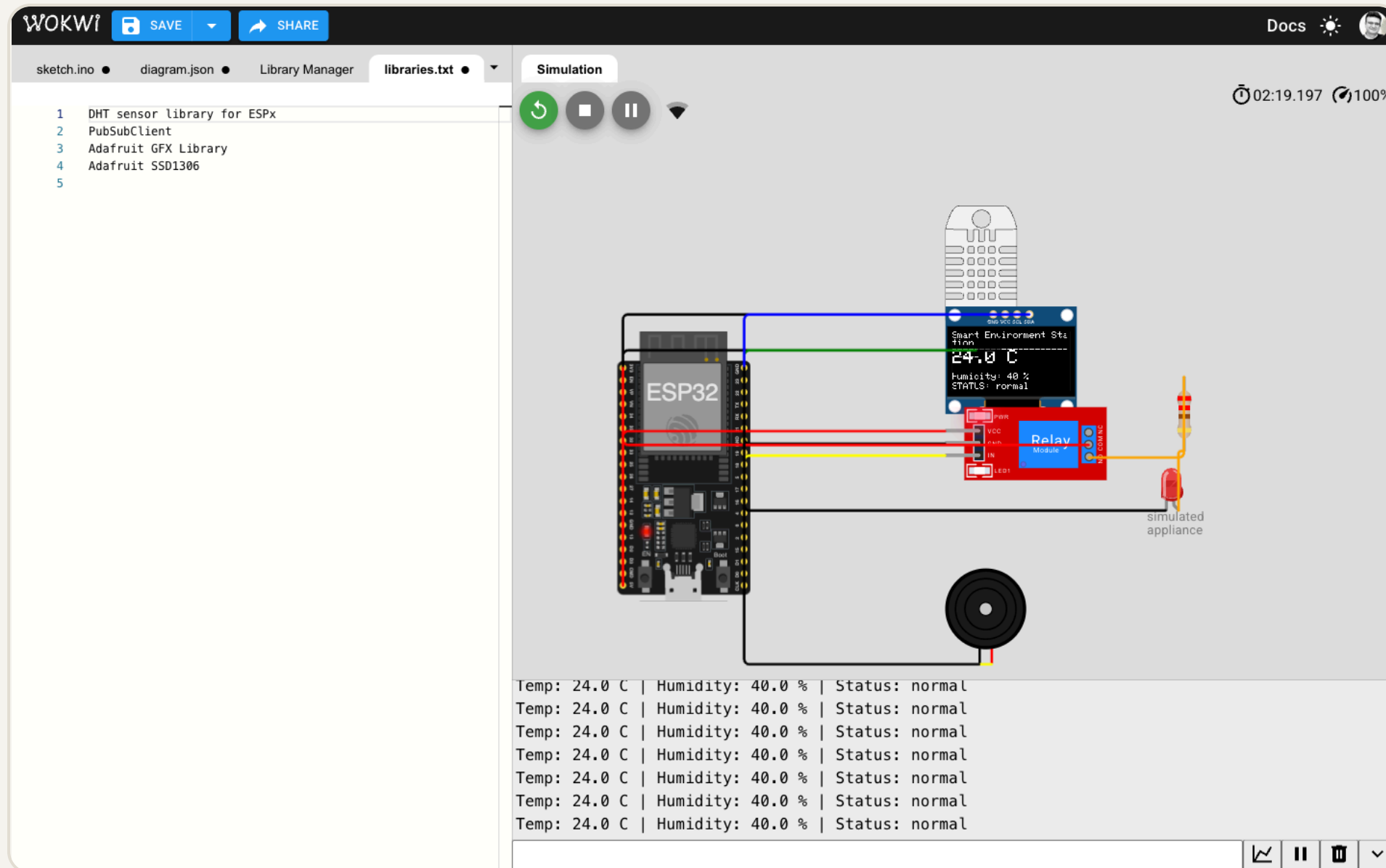
✗ Myth *The most advanced technology gives the best result.*

✓ Fact The simplest design that meets the needs.

✗ Myth *Once devices are installed, the project is done.*

✓ Fact Deployment is the start of years of care.

Capstone: Smart Environment Station



- 1 Open a new ESP32 project in Wokwi
- 2 Paste sketch.ino, and change **YOUR_NAME**
- 3 Paste diagram.json, then add libraries.txt
- 4 Press Play. The first build is slow: be patient
- 5 Push the DHT22 above **28 °C**: watch it react



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

PROJECT SHOWCASE

Present your project in two minutes



1. The problem

“This station protects a room from overheating.”



2. The layers

Point to devices, connectivity, edge, cloud, app.



3. The demo

Above 28 °C: OLED alarm, relay, buzzer, MQTT.



4. One fix

What would you change before trusting it?

Show it to someone next to you, or record a short video.

KEY TAKEAWAYS

What to remember

- ✔ Start with **the problem and the people**, then requirements
- ✔ Every good design choice **traces back to a requirement**
- ✔ Homes, industry, farms: **same five layers**, different choices
- ✔ Prototype up the ladder; test the **riskiest idea** first
- ✔ A PoC must be both **possible** and **worth doing**
- ✔ Plan for power, signal, scale, weather, data, cost... and **people**

Quick quiz

Q1 Why should an IoT design start with the problem, not a technology?

Q2 In the irrigation example, why LoRaWAN instead of Wi-Fi?

Q3 Put in order: pilot, simulation, production, proof of concept, breadboard.

Q4 Sensors work; the farm saves no water. Did the PoC succeed?

THANK YOU

You built a real IoT system. Keep going!

Next steps: keep experimenting in Wokwi, try real hardware, secure your project, explore Node-RED or Home Assistant, and go deeper on one layer.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop